

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



HỌC PHẦN: Xử Lý Ngôn Ngữ Tự Nhiên

BÁO CÁO ĐỀ TÀI MÔN HỌC

**ĐỀ TÀI : DỊCH MÁY ANH-PHÁP VỚI MÔ HÌNH ENCODER-DECODER
LSTM**

Giảng viên hướng dẫn :PGS.TS. Nguyễn Tuấn Đăng

Sinh viên thực hiện:

Võ Gia Kiệt - 3123410182

Phan Đức Nhân - 3123410243

TP. HCM THÁNG 12/2025

[illegible]

Lời Cảm Ơn

Lời đầu tiên, em xin trân trọng gửi lời cảm ơn sâu sắc đến Quý Thầy Cô Trường Đại học Sài Gòn, đặc biệt là các Thầy Cô Khoa Công nghệ Thông tin đã luôn tận tâm giảng dạy và tạo điều kiện thuận lợi để em học tập, rèn luyện trong suốt thời gian qua.

Em xin gửi lời cảm ơn đặc biệt đến Thầy Nguyễn Tuấn Đăng – Giảng viên hướng dẫn môn Xử Lý Ngôn Ngữ Tự Nhiên. Trong suốt quá trình thực hiện đề tài, Thầy đã luôn tận tình giảng dạy, cung cấp những kiến thức quý báu và đưa ra những định hướng quan trọng giúp em hoàn thành tốt bài báo cáo này.

Mặc dù đã cố gắng hết sức trong quá trình nghiên cứu và thực hiện, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Em rất mong nhận được những góp ý quý báu từ Thầy để có thể hoàn thiện bản thân và cải thiện chất lượng trong các dự án tương lai.

Một lần nữa, em xin chân thành cảm ơn Thầy!

Danh sách thuật ngữ và các từ viết tắt

Từ viết tắt	Thuật ngữ đầy đủ (Tiếng Anh)	Giải thích (Tiếng Việt)
NLP	Natural Language Processing	Xử lý Ngôn ngữ Tự nhiên – lĩnh vực nghiên cứu cách máy tính hiểu, xử lý và sinh ngôn ngữ con người
MT	Machine Translation	Dịch máy – bài toán tự động dịch văn bản giữa các ngôn ngữ
NMT	Neural Machine Translation	Dịch máy sử dụng mạng nơ-ron
Seq2Seq	Sequence-to-Sequence	Kiến trúc ánh xạ chuỗi đầu vào sang chuỗi đầu ra
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy xử lý dữ liệu chuỗi
LSTM	Long Short-Term Memory	Biến thể của RNN có khả năng ghi nhớ dài hạn
Bi-LSTM	Bidirectional Long Short-Term Memory	LSTM hai chiều (trái → phải và phải → trái)
Encoder	Encoder Network	Mạng mã hóa câu nguồn thành biểu diễn ngữ nghĩa
Decoder	Decoder Network	Mạng giải mã sinh câu đích từng token
Attention	Attention Mechanism	Cơ chế chú ý giúp mô hình tập trung vào các phần quan trọng của câu nguồn
Luong Attention	Luong-style Attention	Cơ chế attention dựa trên dot-product do Luong et al. đề xuất
Context Vector	Context Vector	Vector ngữ cảnh tổng hợp thông tin

		câu nguồn
h_t	Hidden State	Trạng thái ẩn của LSTM tại thời điểm t
c_t	Cell State	Trạng thái bộ nhớ dài hạn của LSTM
BLEU	Bilingual Evaluation Understudy	Thước đo tự động đánh giá chất lượng dịch máy
BP	Brevity Penalty	Hệ số phạt độ dài trong BLEU
n-gram	n-gram	Cụm gồm n từ liên tiếp trong câu
OOV	Out-Of-Vocabulary	Từ nằm ngoài từ điển huấn luyện
BPE	Byte Pair Encoding	Phương pháp tách từ theo đơn vị subword
PAD	Padding Token	Token dùng để đệm chuỗi
<pad>	Padding Token	Ký hiệu đệm chuỗi
<sos>	Start of Sentence	Ký hiệu bắt đầu câu
<eos>	End of Sentence	Ký hiệu kết thúc câu
<unk>	Unknown Token	Ký hiệu từ không có trong từ điển
TOP_K	Top-K Vocabulary	Số lượng từ tối đa trong vocabulary
MIN_FREQ	Minimum Frequency	Ngưỡng tần suất tối thiểu của từ
stoi	String-to-Index	Ánh xạ từ $\rightarrow$ chỉ số
itos	Index-to-String	Ánh xạ chỉ số $\rightarrow$ từ
GPU	Graphics Processing Unit	Bộ xử lý đồ họa dùng để tăng tốc tính toán
CPU	Central Processing Unit	Bộ xử lý trung tâm
VRAM	Video Random Access Memory	Bộ nhớ của GPU
Adam	Adaptive Moment	Thuật toán tối ưu hóa gradient

	Estimation	
Loss	Loss Function	Hàm mất mát
CrossEntropyLoss	Cross Entropy Loss	Hàm mất mát cho bài toán phân loại
Teacher Forcing	Teacher Forcing	Kỹ thuật dùng token thật khi huấn luyện Decoder
Beam Search	Beam Search Decoding	Chiến lược giải mã xét nhiều giả thuyết
Greedy Decoding	Greedy Decoding	Giải mã tham lam, chọn token tốt nhất tại mỗi bước
Exposure Bias	Exposure Bias	Sai lệch giữa huấn luyện và suy luận
Early Stopping	Early Stopping	Kỹ thuật dừng huấn luyện sớm
Multi30K	Multi30K Dataset	Bộ dữ liệu song ngữ mô tả ảnh
Epoch	Epoch	Một vòng huấn luyện qua toàn bộ tập dữ liệu
Batch	Batch	Một nhóm mẫu dữ liệu huấn luyện
Inference	Inference	Quá trình suy luận/dự đoán
Perplexity	Perplexity	Chỉ số đo mức độ bất định của mô hình ngôn ngữ
Colab	Google Colaboratory	Nền tảng điện toán đám mây của Google

Mục Lục

PHẦN I. MỞ ĐẦU.....	10
1. Giới thiệu đề tài.....	10
2. Mục đích – mục tiêu của đề tài.....	10
2.1. Mục đích.....	10
2.2. Mục tiêu cụ thể.....	10
3. Yêu cầu đồ án.....	11
4. Thư viện sử dụng.....	11
PHẦN II. CƠ SỞ LÝ THUYẾT.....	12
1. Tổng quan về dịch máy (Machine Translation).....	12
2. Kiến trúc Encoder–Decoder.....	12
2.1 Cơ chế context vector.....	12
2.2 Vai trò của hidden state và cell state.....	13
2.2.1. Hidden State (ht) - Trạng thái ẩn:.....	13
2.2.2. Cell State (ct) - Trạng thái tế bào:.....	14
2.2.3. Quá trình chuyển giao (Transfer):.....	14
3. LSTM.....	14
3.1 Công thức.....	14
3.2 Input gate, forget gate, output gate.....	15
4. BLEU Score.....	15
4.1 Công thức.....	16
4.2 Giải thích n-gram matching.....	16
4.2.1. Định nghĩa n-gram:.....	16
4.2.2. Cơ chế Modified Precision (Độ chính xác hiệu chỉnh):.....	16
4.2.3. Tại sao cần Brevity Penalty (BP)?.....	17
PHẦN III. XỬ LÝ DỮ LIỆU.....	17
1. Tập dữ liệu Anh Pháp.....	17
2. Tokenization (SpaCy).....	18
3. Xây dựng Vocabulary.....	19
4. Chuyển câu thành chuỗi id.....	20
5. Padding & Packing.....	20
6. Xây dựng DataLoader.....	22
PHẦN IV. Xây dựng mô hình.....	22
1. Kiến trúc tổng thể.....	22
2. Cấu trúc mô hình.....	24
PHẦN V. HUẤN LUYỆN MÔ HÌNH.....	35
1. Cấu trúc phần cứng (CPU/GPU - Colab).....	35

2. Khởi tạo mô hình và tham số huấn luyện.....	35
3. Hàm mất mát và thuật toán tối ưu.....	36
4. Quy trình huấn luyện.....	36
5. Đánh giá kết quả huấn luyện.....	37
PHẦN VI. ĐÁNH GIÁ MÔ HÌNH VÀ DỰ ĐOÁN.....	39
1. Quy trình suy luận (Inference Process).....	39
2. Xử lý <sos> và <eos>.....	43
3. Hàm translate(sentence).....	44
4. Đánh giá BLEU Score.....	44
5. Ví dụ dịch và phân tích lỗi.....	45
a. Ví dụ đúng.....	45
b. Ví dụ sai.....	48
c. Phân loại lỗi & Giải thích nguyên nhân.....	52
d. Đề xuất cải tiến.....	52
PHẦN VII. KẾT LUẬN, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN.....	54
7.1 Tổng kết đồ án (Summary).....	54
7.2 Đánh giá tổng quan kết quả (Overall Evaluation).....	55
7.3 Hướng phát triển (Future Work).....	56
PHẦN VIII. TÀI LIỆU THAM KHẢO.....	57
PHỤ LỤC.....	58
A. Mã nguồn chính (Source Code Structure).....	58
B. Toàn bộ Notebook.....	66
C. Log Training (Nhật ký huấn luyện).....	66
D. Checkpoint mô hình.....	67
QUY TRÌNH KIỂM THỬ (INFERENCE) MÔ HÌNH DỊCH MÁY ANH-PHÁP.....	67

PHẦN I. MỞ ĐẦU

1. Giới thiệu đề tài

Trong những năm gần đây, bài toán dịch máy (Machine Translation – MT) trở thành một trong những ứng dụng quan trọng nhất của lĩnh vực Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing – NLP). Các hệ thống dịch máy hiện đại thường dựa trên các mô hình học sâu, trong đó kiến trúc Encoder–Decoder sử dụng LSTM là nền tảng quan trọng trước khi các mô hình Transformer xuất hiện.

Đề án này tập trung vào việc **xây dựng từ đầu một mô hình dịch máy Anh–Pháp** sử dụng kiến trúc **Encoder–Decoder LSTM** với **context vector cố định**, không sử dụng các thư viện seq2seq có sẵn như *torchtext.legacy* hoặc *transformers*.

Sinh viên sẽ tự xử lý dữ liệu, xây dựng mô hình, huấn luyện, đánh giá và phân tích kết quả dịch, từ đó hiểu rõ cơ chế hoạt động của mô hình seq2seq truyền thống.

2. Mục đích – mục tiêu của đề tài

2.1. Mục đích

Đề tài nhằm giúp sinh viên hiểu sâu về cách hoạt động của mô hình Encoder–Decoder dựa trên LSTM, thông qua việc tự xây dựng toàn bộ pipeline dịch máy từ tiền xử lý dữ liệu, xây mô hình, huấn luyện và đánh giá.

2.2. Mục tiêu cụ thể

- Xây dựng mô hình Encoder và Decoder bằng LSTM từ đầu.
- Xử lý dữ liệu song ngữ Anh–Pháp: tokenization, vocabulary, padding, batching.
- Huấn luyện mô hình dịch máy bằng PyTorch và theo dõi train/validation loss
- Xây dựng hàm `translate()` cho phép dịch câu tiếng Anh bất kỳ sang tiếng Pháp.
- Đánh giá chất lượng mô hình bằng BLEU score.

- Phân tích lỗi dịch và đề xuất hướng cải tiến như attention hoặc beam search.

3. Yêu cầu đồ án

- Triển khai mô hình Encoder–Decoder LSTM hoàn chỉnh, không dùng thư viện seq2seq có sẵn.
- Xử lý dữ liệu đúng chuẩn, gồm tokenization, xây dựng từ điển, padding, pack_padded_sequence và DataLoader tùy chỉnh.
- Huấn luyện mô hình trên dataset Multi30K (English–French) gồm 29k câu train, 1k câu validation, 1k câu test.
- Lưu checkpoint mô hình tốt nhất (.pth).
- Cài đặt hàm translate(sentence: str) hoạt động chính xác.
- Đánh giá mô hình bằng BLEU score và phân tích ít nhất 5 ví dụ dịch đúng/sai.

4. Thư viện sử dụng

a. Os - Random - Time – Counter

- OS được sử dụng để quản lý đường dẫn dữ liệu, kiểm tra file và tải dữ liệu huấn luyện.
- Random dùng để cố định seed giúp quá trình huấn luyện có tính tái lập.
- Time hỗ trợ đo thời gian huấn luyện từng epoch và tổng thời gian chạy mô hình.
- Counter được dùng để thống kê tần suất xuất hiện của mỗi từ nhằm xây dựng từ điển và giới hạn vocab.

b. spaCy

- Thư viện được sử dụng để tách từ (tokenize) tiếng Anh và tiếng Pháp với tốc độ nhanh và độ chính xác cao.
- Hỗ trợ chuẩn hóa văn bản, loại bỏ ký tự không cần thiết và chuẩn hóa token phục vụ quá trình huấn luyện mô hình.

c. NLTK

- Cung cấp hàm corpus_bleu để tính chỉ số BLEU, đánh giá chất lượng câu dịch của mô hình so với câu tham chiếu.
- Giúp tiêu chuẩn hóa quá trình đánh giá mô hình theo các tiêu chí phổ biến trong dịch máy (Machine Translation).

d. PyTorch

- torch dùng để tạo tensor cho dữ liệu chuỗi và thực hiện các phép toán GPU tăng tốc.
- torch.nn dùng để xây dựng kiến trúc mô hình Encoder – Decoder LSTM, tầng Embedding và tầng dự đoán từ vựng.
- torch.optim (Adam) được sử dụng để tối ưu trọng số trong quá trình huấn luyện.
- DataLoader giúp chia dữ liệu thành batch, xáo trộn dữ liệu và quản lý quá trình nạp dữ liệu.
- Các hàm như pad_sequence, pack_padded_sequence, pad_packed_sequence được dùng để xử lý chuỗi có độ dài khác nhau, giúp LSTM hoạt động chính xác hơn.

e. Matplotlib

- Dùng để vẽ biểu đồ loss theo từng epoch nhằm theo dõi mức độ học của mô hình.
- Hỗ trợ trực quan hóa kết quả dự đoán, ví dụ so sánh câu dịch mô hình với câu tham chiếu.
- Giúp phân tích xu hướng huấn luyện và đánh giá tính ổn định của mô hình.

PHẦN II. CƠ SỞ LÝ THUYẾT

1. Tổng quan về dịch máy (Machine Translation)

Trong bối cảnh toàn cầu hóa và giao tiếp đa ngôn ngữ ngày càng phổ biến, nhu cầu chuyển đổi thông tin giữa các ngôn ngữ trở nên cấp thiết hơn bao giờ hết. Machine Translation (MT) xuất hiện như một giải pháp tự động hỗ trợ con người dịch văn bản từ ngôn ngữ này sang ngôn ngữ khác một cách nhanh chóng và tiết kiệm chi phí.

Nhờ sự phát triển mạnh mẽ của Trí tuệ nhân tạo và các mô hình học sâu, chất lượng dịch máy đã cải thiện đáng kể so với các phương pháp truyền thống. MT hiện không chỉ dừng lại ở mức hỗ trợ tham khảo mà còn đóng vai trò quan trọng trong nhiều ứng dụng thực tế như giao tiếp quốc tế, giáo dục, thương mại điện tử và xử lý dữ liệu đa ngôn ngữ.

2. Kiến trúc Encoder–Decoder

2.1 Cơ chế context vector

Trong kiến trúc Sequence-to-Sequence (Seq2Seq), Context Vector (vector ngữ cảnh) đóng vai trò là cầu nối duy nhất để truyền tải thông tin từ Encoder sang Decoder.

- **Định nghĩa:** Context vector là biểu diễn nén (compressed representation) của toàn bộ câu nguồn (tiếng Anh) dưới dạng một vector số thực có kích thước cố định.
- **Cơ chế hoạt động:**
 1. Mô hình Encoder (LSTM) đọc lần lượt từng token của câu đầu vào. Tại mỗi bước thời gian t , trạng thái của Encoder được cập nhật dựa trên từ hiện tại và trạng thái trước đó.
 2. Sau khi xử lý token cuối cùng của câu nguồn, trạng thái nội tại của Encoder tại bước thời gian cuối cùng ($t=T$) được coi là chứa đựng toàn bộ ý nghĩa, ngữ pháp và ngữ cảnh của câu.
 3. Trạng thái cuối cùng này chính là Context Vector.
- **Trong đồ án này:** Theo yêu cầu đề tài, chúng tôi triển khai **Context Vector cố định**. Điều này có nghĩa là Decoder chỉ nhận thông tin một lần duy nhất từ trạng thái cuối cùng của Encoder để khởi tạo trạng thái đầu tiên của mình, thay vì sử dụng cơ chế Attention tại mỗi bước thời gian.

$$\text{Context Vector} = (h_n(\text{encoder}), c_n(\text{encoder}), a_t)$$

- **Thách thức:** Do kích thước vector là cố định (fixed-length), mô hình có thể gặp khó khăn trong việc nén thông tin của các câu quá dài (Bottleneck problem), dẫn đến hiện tượng quên thông tin ở đầu câu nguồn khi truyền sang Decoder.

2.2 Vai trò của hidden state và cell state

Khác với mạng RNN truyền thống chỉ có một trạng thái ẩn, kiến trúc LSTM (Long Short-Term Memory) sử dụng hai trạng thái riêng biệt để kiểm soát luồng thông tin và giải quyết vấn đề triệt tiêu đạo hàm (vanishing

gradient). Trong mô hình dịch máy này, hai trạng thái đóng vai trò cụ thể như sau:

2.2.1. Hidden State (h_t) - Trạng thái ẩn:

- **Vai trò:** Đại diện cho "trí nhớ ngắn hạn" (short-term memory) hoặc đầu ra của mạng tại thời điểm hiện tại. h_t chứa các thông tin đã được xử lý và lọc qua các cổng (gates) của LSTM, sẵn sàng để đưa ra dự đoán hoặc truyền sang lớp tiếp theo.
- **Trong Encoder:** h_t được cập nhật qua từng từ tiếng Anh. h_n (hidden state cuối cùng) chứa thông tin tóm tắt ngữ nghĩa của cả câu.
- **Trong Decoder:** h_t được dùng trực tiếp để tính toán xác suất từ vựng đầu ra $p(y_t)$ thông qua lớp Linear và hàm Softmax:

$$p(y_t) = \text{softmax}(\text{Linear}(h_t))$$

2.2.2. Cell State (c_t) - Trạng thái tế bào:

- **Vai trò:** Đại diện cho "trí nhớ dài hạn" (long-term memory). Cell state hoạt động như một băng chuyền thông tin chạy xuyên suốt chuỗi dữ liệu với các tương tác tuyến tính nhỏ. Điều này giúp LSTM có khả năng lưu giữ thông tin quan trọng từ đầu câu đến cuối câu mà không bị biến đổi quá nhiều.
- **Cơ chế:** Các cổng (Forget gate, Input gate) quyết định thông tin nào được thêm vào hoặc xóa bỏ khỏi c_t , giúp mô hình học được sự phụ thuộc xa (long-term dependencies) – yếu tố quan trọng trong dịch thuật ngữ pháp phức tạp.

2.2.3. Quá trình chuyển giao (Transfer):

Trong đồ án này, cặp trạng thái cuối cùng của Encoder được sao chép nguyên vẹn để làm trạng thái khởi tạo cho Decoder

3. LSTM

Mạng Long Short-Term Memory (LSTM) là một biến thể đặc biệt của mạng nơ-ron hồi quy (RNN), được thiết kế để giải quyết vấn đề triệt tiêu đạo hàm (vanishing gradient) thường gặp trong RNN truyền thống khi xử lý các chuỗi dữ liệu dài. Trong bài toán dịch máy này, chúng tôi sử dụng LSTM theo cấu trúc Encoder-Decoder như yêu cầu của đề tài. Điểm cốt lõi của LSTM là khả

năng học được sự phụ thuộc xa (long-term dependencies) thông qua cơ chế cell state và các cổng điều hướng thông tin.

3.1 Công thức

Tại mỗi bước thời gian t , LSTM nhận đầu vào là từ hiện tại x_t (dưới dạng embedding vector) và trạng thái từ bước trước đó (h_{t-1} , c_{t-1}). Các tính toán bên trong một tế bào LSTM được thực hiện thông qua các phương trình sau:

1. Forget Gate (Cổng quên):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. Input Gate (Cổng vào):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

3. Cập nhật Cell State:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

4. Output Gate (Cổng ra) & Hidden State:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

Trong đó :

- σ : Hàm kích hoạt Sigmoid (đầu ra trong khoảng $[0, 1]$).
- $\tanh$: Hàm kích hoạt Hyperbolic Tangent (đầu ra trong khoảng $[-1, 1]$).
- W, b : Các ma trận trọng số và bias tương ứng cho từng cổng.
- $\odot$: Phép nhân từng phần tử (Hadamard product).
- $[h_{t-1}, x_t]$: Phép nối (concatenation) giữa hidden state trước đó và input hiện tại.

3.2 Input gate, forget gate, output gate

Các cổng trong LSTM đóng vai trò như những "bộ lọc" thông minh, quyết định lượng thông tin nào được phép đi qua.

4. BLEU Score

Để đánh giá hiệu suất của mô hình dịch máy, đề án sử dụng thang đo **BLEU (Bilingual Evaluation Understudy)**. Đây là thước đo tiêu chuẩn vàng trong các bài toán dịch máy, đo lường sự tương đồng giữa câu dự đoán của mô hình (Candidate) và câu nhãn đúng (Reference)

4.1 Công thức

BLEU Score được tính toán dựa trên độ chính xác của các n-gram (n-gram precision) có trọng số, kết hợp với một hệ số phạt độ dài (Brevity Penalty - BP) để tránh việc mô hình gian lận bằng cách sinh ra các câu quá ngắn.

Công thức tổng quát của BLEU được định nghĩa như sau:

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

Trong đó:

- N : Số lượng n-gram được xem xét (thường $N = 4$ để tính BLEU-4).
- w_n : Trọng số cho từng n-gram (thường là đồng đều $w_n = 1/N$).
- p_n : Độ chính xác (Precision) của n-gram đã được hiệu chỉnh (modified n-gram precision).
- BP : Hệ số phạt độ dài (Brevity Penalty), được tính như sau:

$$BP = \begin{cases} 1 & \text{n\#\# } c > r \\ e^{(1-r/c)} & \text{n\#\# } c \leq r \end{cases}$$

(Với c là độ dài câu dự đoán và r là độ dài câu tham chiếu).

4.2 Giải thích n-gram matching

Cốt lõi của BLEU là việc so sánh các cụm từ (n-gram) xuất hiện trong câu dự đoán với câu tham chiếu.

4.2.1. Định nghĩa n-gram:

- **1-gram (Unigram):** Xét từng từ đơn lẻ. Đo lường mức độ **đầy đủ** (adequacy) của thông tin (câu dịch có chứa các từ khóa đúng không?).
- **2-gram, 3-gram, 4-gram:** Xét các cụm 2, 3, 4 từ liên tiếp. Đo lường độ **trôi chảy** (fluency) và cấu trúc ngữ pháp (các từ có được sắp xếp đúng trật tự không?).

4.2.2. Cơ chế Modified Precision (Độ chính xác hiệu chỉnh):

Một vấn đề của độ chính xác thông thường là mô hình có thể "spam" một từ đúng nhiều lần để đạt điểm cao.

- *Ví dụ:*
 - Reference: "The cat is on the mat."
 - Candidate (Bad): "The the the the the."
 - Nếu tính n-gram (1-gram) thông thường: Độ chính xác là 5/5 (100%) vì "the" có trong Reference.
- **Giải pháp của BLEU:** Sử dụng cơ chế "Clipped Counts" (Đếm cắt ngọn). Số lần xuất hiện của một n-gram trong câu dự đoán sẽ bị giới hạn bởi số lần xuất hiện tối đa của n-gram đó trong câu tham chiếu.
 - Trong ví dụ trên: Từ "the" xuất hiện trong Reference tối đa 2 lần. Do đó, dù Candidate có 5 từ "the", ta chỉ tính là đúng 2 lần.
 - Modified Precision = $2/5 = 40\%$.

4.2.3. Tại sao cần Brevity Penalty (BP)?

Cơ chế matching n-gram có xu hướng ưu tiên các câu ngắn (vì dễ đạt độ chính xác cao hơn). BP được thêm vào để phạt nặng các câu dự đoán ngắn hơn so với câu tham chiếu, đảm bảo câu dịch không chỉ đúng mà còn phải đủ ý.

Kết luận: Điểm BLEU nằm trong khoảng $[0, 1]$ (hoặc 0-100). Điểm càng cao chứng tỏ câu dịch càng gần với văn phong của con người.

PHẦN III. XỬ LÝ DỮ LIỆU

1. Tập dữ liệu Anh Pháp

Dữ liệu sử dụng trong đề tài bao gồm ba tập: tập huấn luyện, tập kiểm tra trong quá trình huấn luyện và tập kiểm tra cuối cùng. Các file dữ liệu được đọc bằng một hàm tự viết `read_lines()`, trong đó mỗi dòng được loại bỏ ký tự xuống dòng và mã hóa UTF-8 để đảm bảo xử lý chính xác các ký tự có dấu tiếng Pháp.

```
def read_lines(path):  
    with open(path, 'r', encoding='utf-8') as f:  
        return [line.strip() for line in f]  
  
train_en =  
read_lines(os.path.join("/content/drive/MyDrive/English_French_Machine_Translation_With_Encoder_Decoder_LSTM_Model/data/train.en"))  
train_fr =  
read_lines(os.path.join("/content/drive/MyDrive/English_French_Machine_Translation_With_Encoder_Decoder_LSTM_Model/data/train.fr"))  
val_en =  
read_lines(os.path.join("/content/drive/MyDrive/English_French_Machine_Translation_With_Encoder_Decoder_LSTM_Model/data/val.en"))  
val_fr =  
read_lines(os.path.join("/content/drive/MyDrive/English_French_Machine_Translation_With_Encoder_Decoder_LSTM_Model/data/val.fr"))  
test_fr =  
read_lines(os.path.join("/content/drive/MyDrive/English_French_Machine_Translation_With_Encoder_Decoder_LSTM_Model/data/test.fr"))  
test_en =  
read_lines(os.path.join("/content/drive/MyDrive/English_French_Machine_Translation_With_Encoder_Decoder_LSTM_Model/data/test.en"))
```

Sau khi đọc dữ liệu, số lượng câu trong từng tập nhằm giúp kiểm tra lại kích thước dữ liệu. Việc tách riêng dữ liệu theo ba nhóm này giúp mô hình được đánh giá khách quan, tránh hiện tượng học vẹt hay overfitting.

```
print("Counts:", len(train_en), len(val_en), len(test_en))

print("Counts:", len(train_fr), len(val_fr), len(test_fr))

Counts: 29000 1014 1071

Counts: 29000 1014 1071
```

2. Tokenization (SpaCy)

Để chuẩn bị cho quá trình huấn luyện, toàn bộ câu trong tập dữ liệu đều được tách thành danh sách token bằng thư viện SpaCy. Đề tài sử dụng mô hình **en_core_web_sm** cho tiếng Anh và **fr_core_news_sm** cho tiếng Pháp. Việc tokenization này giúp đưa câu về dạng tiêu chuẩn thống nhất, loại bỏ sự biến thiên giữa các cách viết khác nhau. Mỗi câu được chuyển về chữ thường (lowercase) để tối ưu dung lượng vocabulary và giảm hiện tượng trùng lặp từ không cần thiết. Đây là bước nền tảng giúp mô hình hiểu rõ cấu trúc của câu trước khi chuyển sang dạng số hóa.

```
nlp_en = spacy.load("en_core_web_sm")
nlp_fr = spacy.load("fr_core_news_sm")

def tokenize_en(text):
    return [tok.text.lower() for tok in nlp_en.tokenizer(text)]

def tokenize_fr(text):
    return [tok.text.lower() for tok in nlp_fr.tokenizer(text)]
```

3. Xây dựng Vocabulary

Sau khi tokenization, tiến hành xây dựng vocabulary dựa trên tần suất xuất hiện từ trong tập huấn luyện. Bằng cách dùng **Counter()**, thống kê số lần xuất hiện của từng từ và lựa chọn tối đa **TOP_K = 10000** từ phổ biến nhất,

đồng thời loại bỏ những từ có tần suất thấp hơn MIN_FREQ. Các token đặc biệt như <pad>, <sos>, <eos>, <unk> cũng được thêm vào ngay từ đầu để mô hình có thể xử lý trường hợp padding, bắt đầu – kết thúc câu và từ hiếm gặp. Từ điển được lưu ở hai dạng ánh xạ: từ sang chỉ số (stoi) và chỉ số sang từ (itos), giúp phục vụ cho cả giai đoạn huấn luyện và suy luận.

```
TOP_K = 10000
MIN_FREQ = 1
SPECIALS = ["<pad>", "<sos>", "<eos>", "<unk>"]

def build_vocab_from_tokenized(sentences_tok, top_k=TOP_K,
min_freq=MIN_FREQ):
    counter = Counter()
    for tokens in sentences_tok:
        counter.update(tokens)

    most = [w for w, c in counter.most_common(top_k) if c >= min_freq]

    stoi = {tok: idx for idx, tok in enumerate(SPECIALS)}

    for w in most:
        if w not in stoi:
            stoi[w] = len(stoi)

    itos = {idx: tok for tok, idx in stoi.items()}

    return stoi, itos
```

4. Chuyển câu thành chuỗi id

Sau khi vocabulary đã được tạo, mỗi câu trong dữ liệu sẽ được chuyển thành một chuỗi số nguyên thông qua hàm numericalize().

```
def numericalize(tokens, stoi, add_sos_eos=False):
    if add_sos_eos:
        seq=[stoi["<sos>"]] + [stoi.get(t, stoi["<unk>"]) for t in tokens] +
[stoi["<eos>"]]
    else:
        seq=[stoi.get(t, stoi["<unk>"]) for t in tokens]
```

Hàm này thay thế mỗi token bằng chỉ số trong vocabulary, đồng thời thêm ký hiệu **<sos>** và **<eos>** vào câu đích (tiếng Pháp) để mô hình biết điểm bắt đầu và kết thúc khi dự đoán. Đối với câu nguồn tiếng Anh, chỉ giữ nguyên chuỗi token mà không thêm token đặc biệt. Việc chuyển đổi này là bước bắt buộc vì mô hình LSTM chỉ làm việc với dữ liệu dạng số, không thể xử lý trực tiếp văn bản.

5. Padding & Packing

Do độ dài các câu khác nhau, hệ thống cần thực hiện padding để đảm bảo mỗi batch có kích thước đồng nhất. Các chuỗi được sắp xếp giảm dần theo độ dài trước khi padding nhằm tối ưu việc sử dụng **pack_padded_sequence()** khi đưa vào LSTM. Điều này giúp mô hình chỉ xử lý thật sự các token hợp lệ thay vì xử lý luôn cả phần padding. Hàm **collate_fn** thực hiện sắp xếp, padding và tạo tensor độ dài chuỗi cho mỗi batch.

```
def collate_fn(batch):
    src_batch, trg_batch = [], []

    for src_tokens, trg_tokens in batch:
        src_batch.append(numericalize(src_tokens, stoi_en, add_sos_eos=False))
        trg_batch.append(numericalize(trg_tokens, stoi_fr, add_sos_eos=True))

    src_lengths = [len(s) for s in src_batch]
    trg_lengths = [len(t) for t in trg_batch]

    sorted_idx = sorted(
        range(len(src_lengths)),
        key=lambda i: src_lengths[i],
        reverse=True
    )

    src_batch = [torch.tensor(src_batch[i], dtype=torch.long) for i in sorted_idx]
    trg_batch = [torch.tensor(trg_batch[i], dtype=torch.long) for i in sorted_idx]

    src_lengths = [src_lengths[i] for i in sorted_idx]
```

```

trg_lengths = [trg_lengths[i] for i in sorted_idx]

src_padded = pad_sequence(
    src_batch,
    batch_first=True,
    padding_value=stoi_en["<pad>"]
)

trg_padded = pad_sequence(
    trg_batch,
    batch_first=True,
    padding_value=stoi_fr["<pad>"]
)

return (
    src_padded.to(DEVICE),
    torch.tensor(src_lengths, dtype=torch.long).to(DEVICE),
    trg_padded.to(DEVICE),
    torch.tensor(trg_lengths, dtype=torch.long).to(DEVICE))

```

6. Xây dựng DataLoader

Cuối cùng, ba đối tượng Dataset gồm train, validation và test được khởi tạo bằng lớp **TranslationDataset**, chứa cặp câu nguồn và câu đích cùng bộ tokenization và từ điển. Mỗi Dataset sau đó được đưa vào **DataLoader** với **batch size 64** cho tập huấn luyện và validation, trong khi tập test sử dụng batch size 1 để đánh giá từng câu riêng biệt. DataLoader kết hợp với **collate_fn** đảm bảo rằng mỗi batch được xử lý đúng chuẩn, sẵn sàng đưa vào mô hình LSTM Encoder–Decoder cho giai đoạn training.

```

train_ds = TranslationDataset(train_en, train_fr, tokenize_en, tokenize_fr,
                              stoi_en, stoi_fr)
val_ds   = TranslationDataset(val_en, val_fr, tokenize_en, tokenize_fr, stoi_en,
                              stoi_fr)
test_ds  = TranslationDataset(test_en, test_fr, tokenize_en, tokenize_fr, stoi_en,
                              stoi_fr)

BATCH_SIZE = 64

```

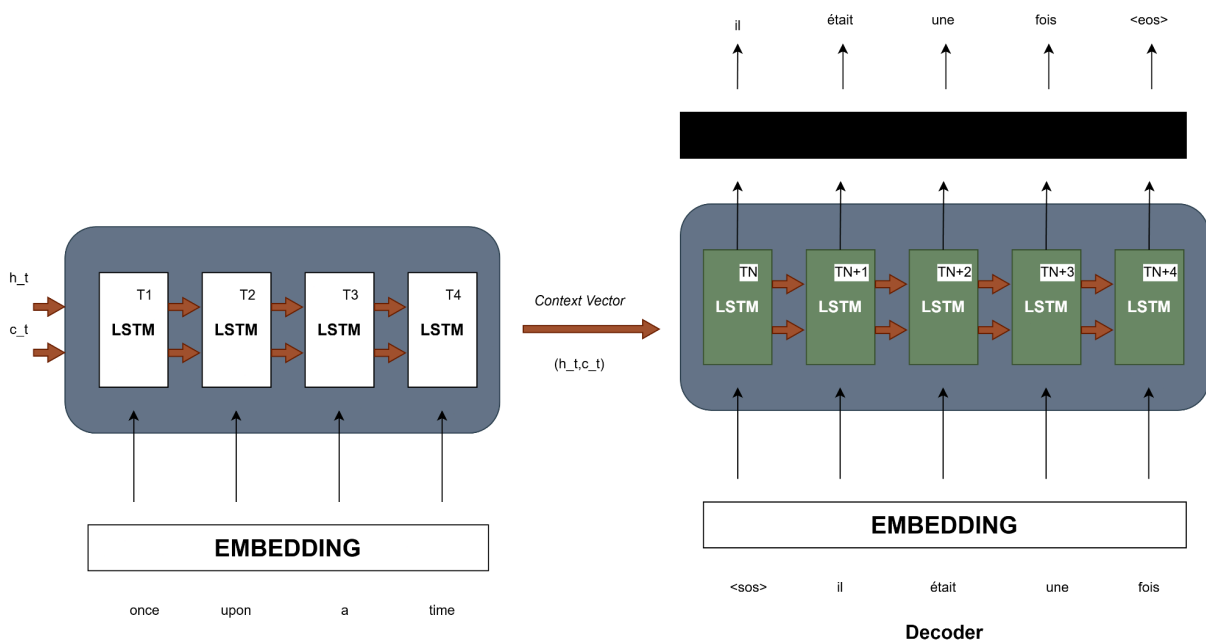
```

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
collate_fn=collate_fn)
val_loader    = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
collate_fn=collate_fn)
test_loader    = DataLoader(test_ds, batch_size=1, shuffle=False,
collate_fn=collate_fn)

```

PHẦN IV. Xây dựng mô hình

1. Kiến trúc tổng thể



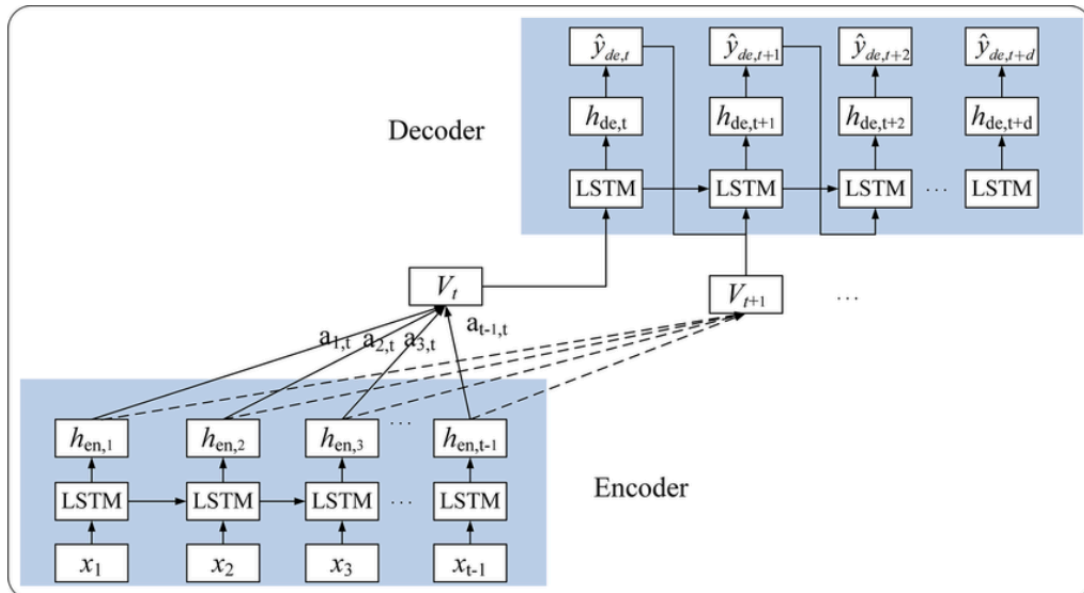
Hình 1. Sơ Đồ Tổng Thể

Kiến trúc Encoder–Decoder LSTM được triển khai trong đồ án tuân theo nguyên lý Seq2Seq truyền thống nhưng được tùy chỉnh để phù hợp với dữ liệu song ngữ Anh–Pháp.

- Encoder LSTM mã hóa toàn bộ câu tiếng Anh thành một cặp trạng thái (h_t , c_t).
- Decoder LSTM được khởi tạo trực tiếp từ cặp trạng thái này và sinh chuỗi tiếng Pháp theo cơ chế teacher forcing trong quá trình huấn luyện.

- Mô hình sử dụng context vector cố định thay vì attention, đúng với yêu cầu bài tập.

2. Cấu trúc mô hình



Hình 2. Sơ Đồ mô hình Luong Attention

Mô hình dịch máy Anh–Pháp được xây dựng theo kiến trúc Encoder–Decoder sử dụng LSTM, kết hợp với Luong Attention để giúp mô hình tập trung vào các từ quan trọng trong câu nguồn khi sinh ra từng từ ở câu đích. Toàn bộ mô hình gồm ba thành phần chính:

- **Attention:** Tính trọng số chú ý giúp mô hình biết từ nào trong câu nguồn quan trọng nhất ở mỗi bước giải mã.
- **Encoder:** Nhận câu tiếng Anh, mã hóa thành chuỗi vector ẩn.
- **Decoder:** Sinh câu tiếng Pháp dựa trên trạng thái ẩn.
- **Seq2Seq:** ghép Encoder và Decoder lại với nhau. Encoder biến câu nguồn thành chuỗi vector ẩn, còn Decoder dùng các vector đó để sinh câu đích từng token một. Seq2Seq đảm bảo dữ liệu chảy đúng giữa hai phần và quản lý toàn bộ quá trình mã hóa $\rightarrow$ giải mã.

a. Lớp Attention

Lớp **LuongAttention** có nhiệm vụ tính toán trọng số chú ý (attention weights) giữa trạng thái ẩn của Decoder và tất cả các vector ẩn do Encoder tạo ra.


```

class LuongAttention(nn.Module):
    def __init__(self, hid_dim):
        super().__init__()
        self.attn = nn.Linear(hid_dim, hid_dim, bias=False)

    def forward(self, decoder_hidden, encoder_outputs, mask):
        dec_hidden = self.attn(decoder_hidden).unsqueeze(2)
        energy = torch.bmm(encoder_outputs, dec_hidden).squeeze(2)
        energy = energy.masked_fill(mask == 0, -1e10)
        attn_weights = torch.softmax(energy, dim=1)

        return attn_weights # [batch, src_len]

```

- `hid_dim`: số chiều vector ẩn của LSTM
- `self.attn` = lớp tuyến tính dùng để biến đổi trạng thái ẩn của decoder.

Hàm `forward(self, decoder_hidden, encoder_outputs, mask)`:

- **decoder_hidden** được chiếu qua layer `self.attn`.
- Thực hiện dot-product giữa các **decoder_hidden** và tất cả các **encoder_outputs** để tìm mức độ tương đồng.
- `mask` giúp bỏ qua vị trí `<pad>`.
- Chuẩn hóa bằng `softmax` để được trọng số.

Hàm **forward** trong lớp **LuongAttention** giúp Decoder xác định **vị trí quan trọng nhất trong câu nguồn** tại mỗi bước dịch. Khi nhận vào trạng thái hiện tại của Decoder (**decoder_hidden**) và toàn bộ các vector do Encoder tạo ra (**encoder_outputs**), hàm sẽ biến `decoder_hidden` thành một dạng phù hợp, rồi so sánh nó với từng vector của Encoder để tính độ khớp. Những vị trí không hợp lệ (**token <pad>**) được loại bỏ bằng `mask`. Cuối cùng, các giá trị độ khớp được chuẩn hóa bằng **softmax** để tạo ra một dãy trọng số, thể hiện mức độ “liên quan” của từng từ nguồn đối với bước dịch hiện tại.

b. Lớp Encoder

Encoder nhận vào câu tiếng Anh (chuỗi token), đưa qua embedding và LSTM để tạo ra:

- **outputs**: vector ẩn tại mỗi timestep
- **hidden, cell**: trạng thái cuối của LSTM

```
class Encoder(nn.Module):
    def __init__(self, input_dim, emb_dim, hid_dim, n_layers=2,
                  dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim,
padding_idx=stoi_en[ "<pad>"])
        self.rnn = nn.LSTM(emb_dim, hid_dim, n_layers, dropout=dropout,
batch_first=True)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, src_lengths):
        embedded = self.dropout(self.embedding(src))
        packed = pack_padded_sequence(embedded, src_lengths.cpu(),
batch_first=True, enforce_sorted=True)
        packed_outputs, (hidden, cell) = self.rnn(packed)
        outputs, _ = pad_packed_sequence(packed_outputs, batch_first=True)
        return outputs, hidden, cell
```

Hàm `forward(self, src, src_lengths)`:

- **input_dim**: số lượng từ trong từ điển tiếng Anh.
- **emb_dim**: số chiều vector embedding.
- **hid_dim**: số chiều vector ẩn của LSTM
- **n_layers**: số tầng LSTM chồng lên nhau.
- **dropout**: tránh overfitting.

Hàm **forward** trong lớp **Encoder** chịu trách nhiệm biến đổi toàn bộ câu tiếng Anh đầu vào thành một chuỗi các vector ẩn (hidden representations) để cung cấp cho Decoder. Khi nhận vào chuỗi token đầu vào (**src**) cùng với độ dài thực tế của từng câu trong batch (**src_lengths**), Encoder sẽ lần lượt đi qua **embedding**, **dropout** và

LSTM để tạo ra biểu diễn ngữ nghĩa của câu nguồn. Cụ thể, hàm forward thực hiện các bước sau:

```
embedded = self.dropout(self.embedding(src))
```

- Khi câu tiếng Anh đầu vào được đưa vào Encoder, đầu tiên nó đi qua **lớp embedding**, nơi mỗi token được chuyển thành một vector liên tục có kích thước **emb_dim**. Lớp embedding này giúp mô hình hiểu được quan hệ giữa các từ ở mức không gian liên tục, thay vì làm việc trực tiếp với chỉ số rời rạc. Các vector embedding sau đó được đưa qua **Dropout** để giảm overfitting bằng cách ngẫu nhiên vô hiệu hóa một số phần tử trong mỗi batch khi huấn luyện.

```
packed = pack_padded_sequence(
    embedded,
    src_lengths.cpu(),
    batch_first=True,
    enforce_sorted=True)
packed_outputs, (hidden, cell) = self.rnn(packed)
```

- Toàn bộ chuỗi embedding được xử lý bằng LSTM nhiều tầng (**n_layers**). Tuy nhiên, vì mỗi câu có độ dài khác nhau, bạn sử dụng **pack_padded_sequence** để mô hình chỉ xử lý những phần thật sự chứa từ, bỏ qua các token **<pad>**. Điều này giúp Encoder không tốn công tính toán dư thừa và tránh việc nhiễu ảnh hưởng đến trạng thái ẩn.

```
outputs, _ = pad_packed_sequence(packed_outputs,
    batch_first=True)
return outputs, hidden, cell
```

- Sau khi xử lý bằng LSTM, Encoder tạo ra hai loại kết quả. **Outputs** là các hidden state tại từng thời điểm, biểu diễn thông tin ngữ nghĩa của mỗi token trong câu. **Hidden và cell** là trạng

thái cuối cùng của các tầng LSTM, chứa phần tóm lược toàn bộ câu nguồn và sẽ được truyền sang Decoder để bắt đầu quá trình dịch. Cuối cùng, hàm **pad_packed_sequence** đưa outputs về đúng dạng có padding như ban đầu để chúng có thể được dùng cho Attention trong Decoder.

Cách hoạt động:

1. Embeddings:

```
embedded = self.dropout(self.embedding(src))
```

Chuyển token thành vector.

2. Pack sequence:

Sử dụng **pack_padded_sequence** để tối ưu tốc độ và tránh tính toán ở <pad>.

3. LSTM

Outputs: toàn bộ output theo từng timestep.

hidden, cell: trạng thái cuối cùng của LSTM, sẽ dùng để khởi tạo Decoder

→ Đây là quá trình mã hóa (encoding) câu nguồn thành biểu diễn ngữ nghĩa.

c. Lớp Decoder

Decoder sinh ra câu đích (tiếng Pháp) từng từ một dựa vào:

- Từ trước đó.
- Trạng thái ẩn hiện tại.
- Context vector

```
class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, hid_dim, n_layers=2,
                  dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(output_dim,
                                       emb_dim,
                                       padding_idx=PAD_IDX)
        self.rnn = nn.LSTM(emb_dim + hid_dim,
                           hid_dim,
                           n_layers,
```

```

        dropout=dropout, batch_first=True)
self.fc_out = nn.Linear(hid_dim * 2, output_dim)
self.dropout = nn.Dropout(dropout)
self.attention = LuongAttention(hid_dim)

def forward(self, input_step, hidden, cell, encoder_outputs, mask):
    input_step = input_step.unsqueeze(1)
    embedded = self.dropout(self.embedding(input_step))

    dec_hidden = hidden[-1]
    attn_weights = self.attention(dec_hidden, encoder_outputs,
mask)
    attn_weights_unsq = attn_weights.unsqueeze(1)

    context = torch.bmm(attn_weights_unsq, encoder_outputs)
    rnn_input = torch.cat((embedded, context), dim=2)

    output, (hidden, cell) = self.rnn(rnn_input, (hidden, cell))
    output = output.squeeze(1)
    context = context.squeeze(1)

    pred = self.fc_out(torch.cat((output, context), dim=1))
    return pred, hidden, cell, attn_weights

```

- Output_dim: kích thước từ vựng tiếng pháp (số lượng từ)
- Emb_dim: số chiều embedding của tiếng pháp
- Hid_dim: số chiều của hidden state trong LSTM
- N_layers: số layer LSTM
- Dropout: tỉ lệ dropout chống overfitting.

Hàm forward(self, input_step, hidden, cell, encoder_outputs, mask):

- Input_step: Token tiếng Pháp ở bước hiện tại (ví dụ: chỉ số từ “je”, “suis”, ...).
- Hidden: Hidden state của LSTM ở bước trước.
- Cell: cell state của LSTM ở bước trước.
- Encoder_outputs: toàn bộ output của encoder.
- Mask: dùng để bỏ qua các token <pad> khi tính attention.

Các bước trong hàm forward của lớp Decoder:

Hàm **forward** trong lớp **Decoder** chịu trách nhiệm sinh ra **một token tiếp theo** trong câu đích tại *mỗi bước thời gian*. Khi nhận vào token hiện tại (**input_step**), trạng thái ẩn từ bước trước (**hidden, cell**), cùng toàn bộ output của Encoder (**encoder_outputs**), Decoder sẽ kết hợp embedding, attention và LSTM để tạo ra dự đoán từ vựng tiếp theo. Cụ thể, hàm thực hiện các bước sau:

```
input_step = input_step.unsqueeze(1)
embedded = self.dropout(self.embedding(input_step))
```

- Token đầu vào được mở rộng thành một chuỗi có chiều dài thời gian bằng 1 (bằng **unsqueeze(1)**), rồi được đưa qua lớp **embedding** để chuyển thành một vector liên tục có kích thước **emb_dim**. Vector embedding này được áp dụng **dropout** nhằm giảm nguy cơ overfitting.

```
dec_hidden = hidden[-1]
attn_weights = self.attention(dec_hidden, encoder_outputs, mask)
attn_weights_unsq = attn_weights.unsqueeze(1)
```

- Decoder lấy trạng thái ẩn của **tầng LSTM cuối cùng** (**dec_hidden = hidden[-1]**) và đưa vào cơ chế **Luong Attention** để so sánh với tất cả vector **encoder_outputs**. Kết quả là một vector **trọng số attention**, đã được chuẩn hóa bằng softmax và đã được áp dụng mask để đảm bảo mô hình không chú ý vào các token **<pad>**.

```
context = torch.bmm(attn_weights_unsq, encoder_outputs)
rnn_input = torch.cat((embedded, context), dim=2)
output, (hidden, cell) = self.rnn(rnn_input, (hidden, cell))
```

- Các trọng số attention này được nhân ma trận (*weighted sum*) với **encoder_outputs** để tạo ra **context vector**, là một vector tổng hợp thể hiện phần quan trọng nhất của câu nguồn tại bước

thời gian hiện tại. Sau đó, Decoder ghép nối vector embedding của token và context vector lại với nhau theo chiều đặc trưng, tạo thành tensor **rnn_input**, rồi truyền vào LSTM để thực hiện **một bước tính toán duy nhất**. LSTM trả về **output tạm thời**, cùng với cặp trạng thái **hidden** và **cell** đã được cập nhật cho bước tiếp theo.

```
output = output.squeeze(1)
context = context.squeeze(1)
pred = self.fc_out(torch.cat((output, context), dim=1))
return pred, hidden, cell, attn_weights
```

- Output tạm thời này được nối lại với context vector rồi đưa qua lớp tuyến tính **fc_out** để tạo ra **vector pred**, tức là điểm số (logits) cho toàn bộ từ vựng tiếng Pháp. Đây là dự đoán của Decoder về token tiếp theo. Hàm trả về **pred**, trạng thái **hidden**, **cell** đã cập nhật và vector **attention** giúp mô hình phân bổ trọng tâm vào các phần khác nhau của câu nguồn.

Các bước xử lý:

1. Chuẩn hóa input

Input_step = input_step.unsqueeze(1).

- Biến input_step từ dạng [batch] thành [batch, 1] vì LSTM cần dữ liệu dạng chuỗi dài.

2. Embedding

embedded = self.dropout(self.embedding(input_step))

- Biến token thành vector độ dài emb_dim.

Ví dụ: input_step = 42, embedding = vector 256 chiều

Kết quả của embedded.shape = [1, 1, 256]

3. Lấy hidden state hiện tại

dec_hidden = hidden[-1]

- Lấy hidden của layer cuối cùng của LSTM

Ví dụ: hidden có dạng [layers, batch, hidden] = [2, 1, 512] thì

dec_hidden = [1, 512]

4. Tính Attention

```
attn_weights = self.attention(dec_hidden, encoder_outputs, mask)
```

- So sánh hidden hiện tại của Decoder với từng output của Encoder.
- Tính điểm Luong score $\rightarrow$ softmax $\rightarrow$ trọng số attention.
Ví dụ: câu nguồn có 4 từ $\Rightarrow$ Attention trả về: [0.1, 0.2, 0.6, 0.3]
 $\Rightarrow$ Mô hình đang tập trung mạnh nhất vào từ thứ 3.

5. Tạo context vector

```
context = torch.bmm(attn_weights_unsq, encoder_outputs)
```

- Nhân attention với encoder_outputs.
- Cho ra context vector 512 chiều
Ví dụ:
encoder_outputs: [1, 5, 512]
attn_weights: [1, 5]
context $\rightarrow$ [1, 1, 512]

6. Sinh từ tiếp theo

```
pred = self.fc_out(torch.cat((output, context), dim=1))
```

- Ghép output + context.
- Đưa qua Linear $\rightarrow$ vector xác suất trên toàn bộ từ điển tiếng Pháp.
Ví dụ: pred là [0.001, 0.0045, 0.56, ..., 0.08] thì từ có xác suất sẽ thành từ tiếp theo .

d. Lớp Seq2Seq

Lớp Seq2Seq đóng vai trò là một lớp điều phối (controller), kết nối Encoder LSTM, Decoder LSTM và cơ chế Attention để tạo thành mô hình dịch tự động hoàn chỉnh. Lớp này quản lý toàn bộ quá trình chạy mô hình theo chuỗi:


```

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device,
trg_pad_idx=PAD_IDX):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device
        self.trg_pad_idx = trg_pad_idx

    def create_mask(self, src):
        return (src != stoi_en["<pad>"]).to(self.device) # [batch, src_len]

    def forward(self, src, src_lengths, trg, teacher_forcing_ratio=0.5):
        batch_size = src.size(0)
        trg_len = trg.size(1)
        trg_vocab_size = self.decoder.fc_out.out_features

        outputs = torch.zeros(
            batch_size, trg_len, trg_vocab_size
        ).to(self.device)

        encoder_outputs, hidden, cell = self.encoder(src, src_lengths)
        mask = self.create_mask(src)

        input_token = trg[:, 0] # <sos>
        for t in range(1, trg_len):
            output, hidden, cell, _ = self.decoder(
                input_token, hidden, cell, encoder_outputs, mask
            )
            outputs[:, t, :] = output
            teacher_force = random.random() < teacher_forcing_ratio
            top1 = output.argmax(1)
            input_token = trg[:, t] if teacher_force else top1

        return outputs

```

Hàm forward(self, src, src_length, trg, teacher_forcing_ratio = 0.5)

- Src: ma trận token câu nguồn (kích thước: batch_size × src_len), các chỉ số từ vựng tiếng Anh.

- `Src_lengths`: chiều dài thực của mỗi câu trong batch (dùng cho `pack_padded_sequence`).
- `Trg`: ma trận token câu mục tiêu (kích thước: `batch_size × trg_len`) — dùng để huấn luyện bằng teacher forcing.
- `Teacher_forcing_ratio`: xác suất dùng từ thực (`trg[:, t]`) làm input bước tiếp theo vì dùng dự đoán mô hình.

Các bước trong hàm forward của lớp `seq2seq`:

Hàm **forward** trong lớp **Seq2Seq** giữ vai trò điều phối toàn bộ quá trình dịch tự động giữa Encoder và Decoder. Khi nhận vào câu nguồn (tiếng Anh) và câu đích đã được dịch dịch chuẩn (tiếng Pháp), hàm này sẽ lần lượt xử lý các bước sinh câu như sau:

```
batch_size = src.size(0)
trg_len = trg.size(1)
trg_vocab_size = self.decoder.fc_out.out_features
outputs = torch.zeros(
    batch_size, trg_len, trg_vocab_size
).to(self.device)
```

- Mô hình lấy **batch_size** của câu nguồn và **trg_len** (độ dài câu đích) để xác định kích thước cần thiết cho đầu ra. Sau đó, một tensor **outputs** chứa toàn số 0 được khởi tạo để lưu lại toàn bộ phân phối xác suất dự đoán cho từng token của câu tiếng Pháp ở mọi bước thời gian.

```
encoder_outputs, hidden, cell = self.encoder(src, src_lengths)
mask = self.create_mask(src)
```

- Câu tiếng Anh được đưa vào **Encoder**, trả về **encoder_outputs** (ma trận biểu diễn toàn bộ câu nguồn), đồng thời cung cấp trạng thái ẩn **hidden** và **cell** để khởi tạo quá trình sinh câu ở Decoder. Hàm cũng tạo ra một **mask** để giúp cơ chế attention bỏ qua các vị trí **<pad>** trong câu nguồn

```

input_token = trg[:, 0] # <sos>
for t in range(1, trg_len):
    output, hidden, cell, _ = self.decoder(
        input_token, hidden, cell, encoder_outputs, mask
    )
    outputs[:, t, :] = output
    teacher_force = random.random() < teacher_forcing_ratio
    top1 = output.argmax(1)
    input_token = trg[:, t] if teacher_force else top1

return outputs

```

- Mô hình bắt đầu sinh câu bằng token **<sos>** của câu đích. Ở mỗi bước, Decoder nhận: token hiện tại, hidden, cell, encoder_outputs và mask. Decoder trả về phân phối xác suất của từ tiếp theo, và kết quả này được lưu vào tensor **outputs**. Sau đó cơ chế **teacher forcing** quyết định token kế tiếp đưa vào Decoder: hoặc token thật từ dữ liệu huấn luyện, hoặc token mô hình tự dự đoán. Quá trình tiếp tục cho đến hết chiều dài câu đích. Kết thúc hàm, mô hình trả về ma trận **outputs** chứa toàn bộ dự đoán của Decoder.

PHẦN V. HUẤN LUYỆN MÔ HÌNH

1. Cấu trúc phần cứng (CPU/GPU - Colab)

Để đáp ứng yêu cầu tính toán của mô hình học sâu sử dụng LSTM, nhóm đã triển khai quá trình huấn luyện trên nền tảng Google Colab nhằm tận dụng khả năng tăng tốc bằng GPU. Việc sử dụng GPU giúp rút ngắn đáng kể thời gian huấn luyện và ổn định quá trình tối ưu tham số.

Cấu hình phần cứng chi tiết:

- **Nền tảng:** Google Colab (Free Tier).
- **GPU:** NVIDIA Tesla T4 (16GB VRAM, 2560 CUDA cores).
- **CPU:** Intel Xeon (2 vCPU @ 2.20GHz).
- **RAM hệ thống:** 12.7 GB.

Cấu hình phần mềm:

- **Ngôn ngữ:** Python 3.10.
- **Deep Learning Framework:** PyTorch
- **NLP Library:** Spacy (Tokenization), TorchText.
- **NLTK** (BLEU Score)
- **Visualization:** Matplotlib. (Trực quan hoá loss)

2. Khởi tạo mô hình và tham số huấn luyện

Mô hình Encoder–Decoder LSTM được khởi tạo với các siêu tham số đã được lựa chọn thông qua thực nghiệm nhằm cân bằng giữa khả năng biểu diễn và chi phí tính toán.

Tham số	Giá trị
Embedding dimension	256
Hidden dimension	512
Số lớp LSTM	2
Dropout	0.3
Batch size	64
Epoch tối đa	20
Learning rate	0.001
Teacher forcing ratio	0.5
Gradient clipping	1.0

3. Hàm mất mát và thuật toán tối ưu

Trong quá trình huấn luyện, hàm mất mát CrossEntropyLoss được sử dụng để đo lường sự sai khác giữa từ dự đoán và từ tham chiếu trong câu tiếng Pháp. Token **<pad>** được loại bỏ khỏi quá trình tính loss nhằm tránh ảnh hưởng đến gradient.

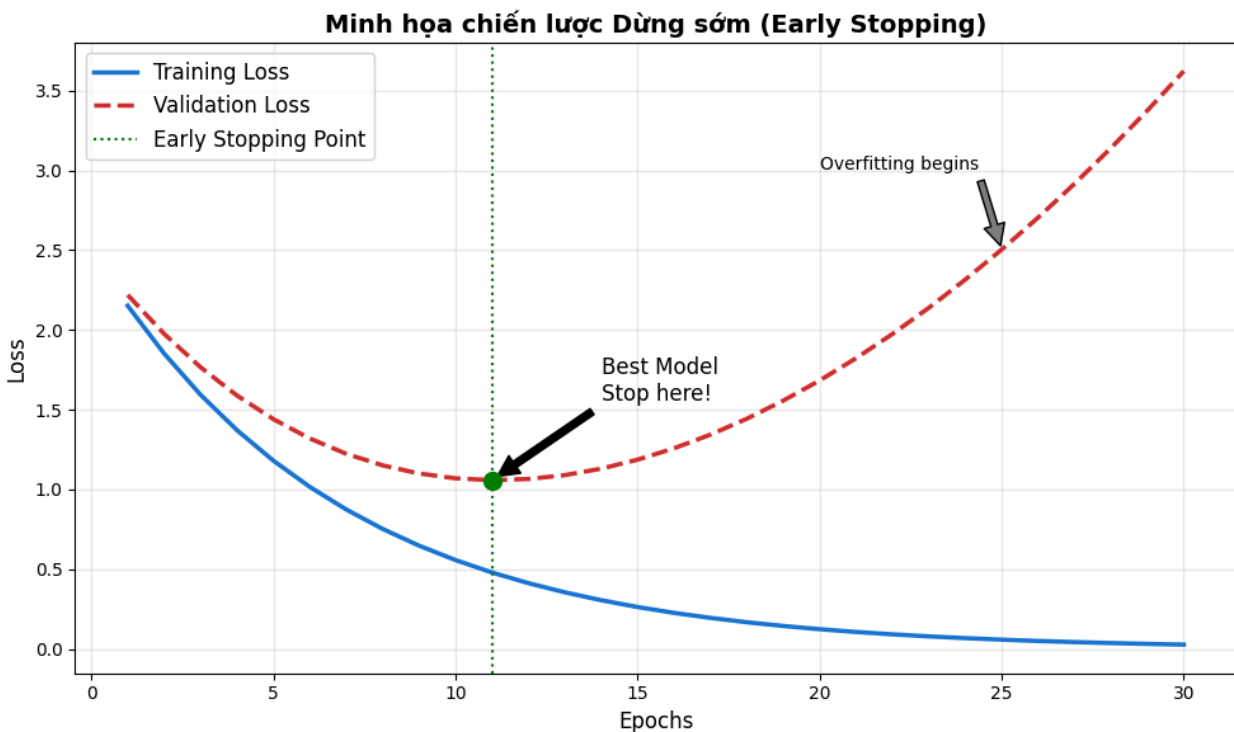
Mô hình được tối ưu bằng thuật toán Adam với learning rate ban đầu là 0.001, giúp tăng tốc hội tụ và ổn định quá trình học.

4. Quy trình huấn luyện

Mỗi epoch huấn luyện bao gồm hai giai đoạn:

- Training phase: Mô hình được huấn luyện với teacher forcing ratio = 0.5 nhằm giúp Decoder học được mối quan hệ ngôn ngữ trong giai đoạn đầu.
- Validation phase: Teacher forcing được tắt (ratio = 0) nhằm đánh giá khả năng suy luận thực tế của mô hình.

Để tránh hiện tượng exploding gradient trong mạng LSTM, kỹ thuật Gradient Clipping với ngưỡng 1.0 được áp dụng. Ngoài ra, chiến lược Early Stopping được sử dụng với patience = 3 nhằm dừng huấn luyện khi val_loss không còn cải thiện.



5. Đánh giá kết quả huấn luyện

a . Kết quả huấn luyện qua từng epoch

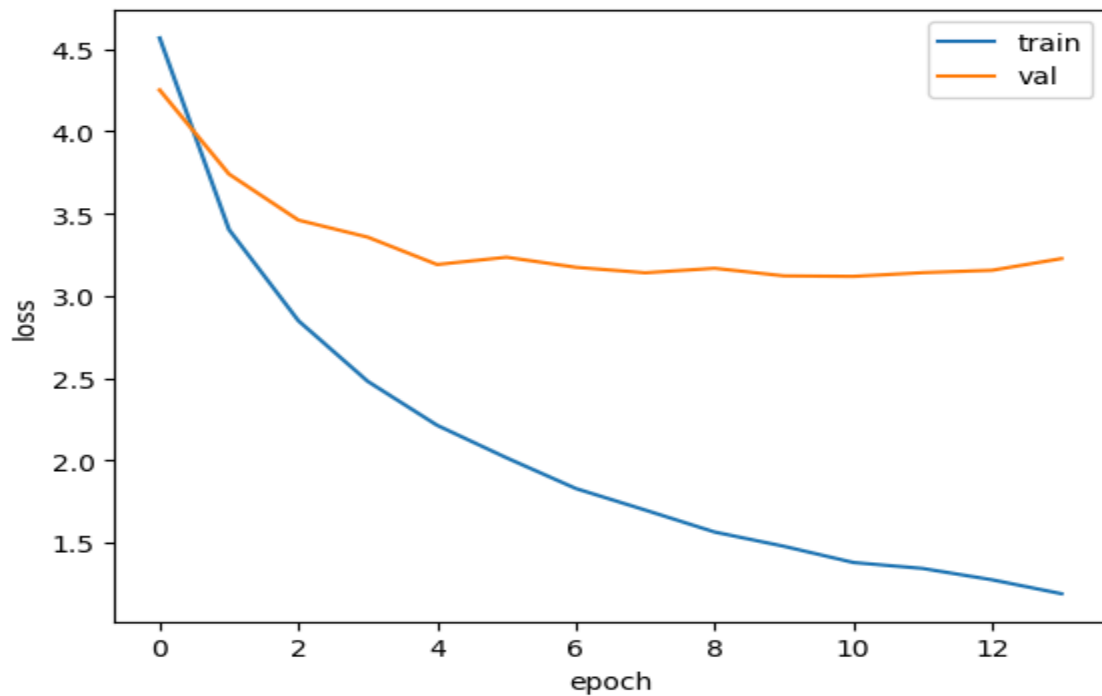
Epoch 1 | Time 1m 34s | Train Loss: 4.5686 | Val Loss: 4.2521

Epoch 2 | Time 1m 34s | Train Loss: 3.4034 | Val Loss: 3.7407
Epoch 3 | Time 1m 34s | Train Loss: 2.8495 | Val Loss: 3.4610
Epoch 4 | Time 1m 35s | Train Loss: 2.4810 | Val Loss: 3.3573
Epoch 5 | Time 1m 34s | Train Loss: 2.2139 | Val Loss: 3.1909
Epoch 6 | Time 1m 34s | Train Loss: 2.0163 | Val Loss: 3.2353
Epoch 7 | Time 1m 35s | Train Loss: 1.8295 | Val Loss: 3.1741
Epoch 8 | Time 1m 35s | Train Loss: 1.6986 | Val Loss: 3.1407
Epoch 9 | Time 1m 35s | Train Loss: 1.5651 | Val Loss: 3.1681
Epoch 10 | Time 1m 34s | Train Loss: 1.4783 | Val Loss: 3.1219
Epoch 11 | Time 1m 35s | Train Loss: 1.3799 | Val Loss: 3.1187
Epoch 12 | Time 1m 34s | Train Loss: 1.3436 | Val Loss: 3.1415
Epoch 13 | Time 1m 34s | Train Loss: 1.2748 | Val Loss: 3.1560
Epoch 14 | Time 1m 34s | Train Loss: 1.1901 | Val Loss: 3.2270

Early stopping triggered.

Ngoài ra, có thể quan sát rằng Validation Loss giảm mạnh trong các epoch đầu (từ 4.25 xuống khoảng 3.12), sau đó gần như không cải thiện đáng kể và dao động nhẹ quanh một ngưỡng nhất định. Điều này cho thấy mô hình đã tiệm cận trạng thái hội tụ. Cơ chế Early Stopping được kích hoạt tại epoch 14 nhằm dừng huấn luyện kịp thời khi val_loss không còn cải thiện, giúp hạn chế hiện tượng overfitting và tiết kiệm tài nguyên tính toán.

b. Biểu đồ hiển thị val_loss



Hình 2. Biểu đồ val_loss

Khoảng cách giữa train_loss và val_loss tăng dần về cuối quá trình huấn luyện là hiện tượng phổ biến trong các mô hình Seq2Seq sử dụng LSTM. Nguyên nhân chủ yếu đến từ giới hạn của context vector cố định và sự khác biệt phân phối giữa tập huấn luyện và validation. Tuy vậy, do val_loss không tăng trở lại mà chỉ dao động nhẹ, có thể kết luận rằng mô hình chưa bị overfitting nghiêm trọng và vẫn duy trì khả năng tổng quát hóa chấp nhận được.

PHẦN VI. ĐÁNH GIÁ MÔ HÌNH VÀ DỰ ĐOÁN

1. Quy trình suy luận (Inference Process)

Sau khi mô hình được huấn luyện, quá trình dự đoán câu tiếng Pháp từ 1 câu Tiếng Anh mới được thực hiện các bước sau :

Bước 1 - Tiền xử lý câu đầu vào

Khi người dùng nhập một câu tiếng Anh, mô hình thực hiện bước tiền xử lý để đưa câu về dạng tensor có thể đưa vào LSTM.

Các bước cụ thể:

1. Nhận câu đầu vào dạng chuỗi (str).
Ví dụ “a man is riding a bicycle”
2. **Tách từ (tokenization)** bằng mô hình spaCy tiếng Anh:

```
tokens = tokenize_en(sentence)
```

→ Trả về danh sách token dạng chữ thường: ["a", "man", "is", "riding", "a", "bicycle"].

3. **Chuyển mỗi token thành chỉ số số học (ID)** dựa trên vocabulary tiếng Anh:

```
src_indices = [stoi_en.get(t, UNK_IDX) for t in tokens]
```

4. **Chuyển thành tensor PyTorch** và reshape về dạng [1, seq_len]:

```
src_tensor =  
torch.tensor(src_indices, dtype=torch.long).unsqueeze(0).to(DEVICE)
```

Điều này cho phép mô hình xử lý từng câu riêng lẻ khi suy luận.

Bước 2 - Mã hoá bằng Encoder

Câu đầu vào dạng tensor được đưa qua Encoder LSTM để trích xuất thông tin ngữ nghĩa.

Quy trình diễn ra như sau:

- Encoder thực hiện embedding → LSTM
- Đồng thời sử dụng **pack_padded_sequence()** để tối ưu xử lý các câu có độ dài khác nhau.
- Kết quả trả về:

```
encoder_outputs, hidden, cell = model.encoder(src_tensor, src_len)
```

Trong đó:

- **encoder_outputs**: ma trận ẩn của toàn bộ các bước thời gian.

- **hidden, cell**: trạng thái cuối cùng của LSTM → đây chính là **context vector cố định** của mô hình Seq2Seq truyền thống.

Trạng thái (**h_n**, **c_n**) này được dùng để khởi tạo Decoder.

Bước 3 - Khởi tạo Decoder

- Token đầu tiên đưa vào Decoder luôn là **<sos>** (Start of Sentence).
- Hidden state và cell state ban đầu của Decoder được **sao chép từ Encoder**:

```
def forward(self, src, src_lengths, trg, teacher_forcing_ratio=0.5):
    batch_size = src.size(0)
    trg_len = trg.size(1)
    trg_vocab_size = self.decoder.fc_out.out_features
    outputs = torch.zeros(batch_size, trg_len, trg_vocab_size).to(self.device)
    encoder_outputs, hidden, cell = self.encoder(src, src_lengths)
    mask = self.create_mask(src)

    input_token = trg[:, 0] # <sos>
    for t in range(1, trg_len):
        output, hidden, cell, _ = self.decoder(input_token, hidden, cell,
encoder_outputs, mask)
        outputs[:, t, :] = output
        teacher_force = random.random() < teacher_forcing_ratio
        top1 = output.argmax(1)
        input_token = trg[:, t] if teacher_force else top1

    return outputs
```

Điều này tạo cầu nối giữa câu nguồn (EN) và câu đích (FR).

Bước 4 - Beam Search Decoding (mô hình cải tiến)

Trong đề án này, nhóm không sử dụng Greedy Decoding cho giai đoạn suy luận cuối cùng mà thay thế bằng **Beam Search** nhằm cải thiện chất lượng câu đích.

Quy trình Beam Search diễn ra như sau:

- Sau khi Encoder xử lý xong câu nguồn, ta thu được **encoder_outputs**, cùng với trạng thái ẩn cuối cùng (**hidden, cell**) và mặt nạ **mask**.
- Beam ban đầu được khởi tạo với **một giả thuyết duy nhất**:
 - chuỗi gồm token bắt đầu câu: [**<sos>**],
 - trạng thái ẩn và cell lấy từ Encoder,
 - điểm số log-probability ban đầu bằng 0.

→ Mỗi phần tử trong beam có dạng: (**sequence, hidden, cell, score**).

Trong mỗi bước thời gian (tối đa **max_len** bước):

1. Tạo một danh sách **new_beams** rỗng để chứa các giả thuyết mở rộng.

```
for _ in range(max_len):
    new_beams = []

    for seq, h, c, score in beams:

        last_token = seq[-1]

        # Nếu đã kết thúc câu
        if last_token == EOS_IDX:
            new_beams.append((seq, h, c, score))
            continue

        input_token = torch.tensor([last_token], dtype=torch.long).to(DEVICE)

        with torch.no_grad():
            output, h_new, c_new, _ = model.decoder(
                input_token, h, c, encoder_outputs, mask
            )

        log_probs = torch.log_softmax(output, dim=1) # [1, vocab]
        topk_log_probs, topk_ids = torch.topk(log_probs, beam_width)

        # Thêm vào beam mới
        for i in range(beam_width):
            new_seq = seq + [topk_ids[0][i].item()]
            new_score = score + topk_log_probs[0][i].item()
            new_beams.append((new_seq, h_new, c_new, new_score))
```

2. Với từng giả thuyết (seq, h, c, score) trong beam hiện tại:

- Lấy **token cuối cùng** $\text{last_token} = \text{seq}[-1]$
- Nếu $\text{last_token} == \langle \text{eos} \rangle$ thì giả thuyết này được giữ nguyên đưa sang new_beams mà không mở rộng thêm (câu đã kết thúc).
- Ngược lại
 - Token cuối cùng được đưa vào **Decoder** cùng với trạng thái (h,c), encoder_outputs và mask
 - Decoder sinh ra phân phối xác suất trên toàn bộ từ vựng tại bước hiện tại.
 - Chọn ra **top-k token** có xác suất cao nhất (với k là beam width).
 - Với mỗi token được chọn:
 - Tạo một giả thuyết mới bằng cách:
 - Nối token đó vào chuỗi hiện tại
 - Cập nhật hidden, cell mới từ Decoder,
 - Cộng dồn điểm số:

$$\text{score_new} = \text{score} + \log P(\text{token})$$

- Thêm giả thuyết mới vào new_beams .

3. Quá trình này lặp lại cho đến khi:

- Tất cả các giả thuyết đều sinh ra token $\langle \text{eos} \rangle$, hoặc
- Đạt đến độ dài câu tối đa cho phép

So với Greedy Decoding (chỉ chọn token tốt nhất tại mỗi bước), Beam Search cho phép mô hình **xem xét đồng thời nhiều phương án dịch**, từ đó hạn chế lỗi do lựa chọn sai token ở những bước đầu và tạo ra câu dịch trôi chảy, hợp ngữ pháp hơn.

Bước 5 - Điều kiện dừng

Quá trình sinh bị dừng trong hai trường hợp:

1. Decoder tạo ra token $\langle \text{eos} \rangle$
2. Vượt quá độ dài tối đa

Sau đó mô hình:

- Bỏ <sos> và <eos>
- Chuyển chuỗi ID → token → câu tiếng Pháp hoàn chỉnh

2. Xử lý <sos> và <eos>

Mô hình Seq2Seq sử dụng 2 token đặc biệt này trong cả huấn luyện và suy luận:

<sos>

- Đánh dấu *bắt đầu câu*
- Là token đầu tiên mà Decoder nhận
- Cho phép mô hình biết khi nào bắt đầu quá trình sinh

<eos>

- Đánh dấu *kết thúc câu*
- Khi mô hình tạo ra <eos> → inference dừng
- Tránh sinh chuỗi vô hạn và làm BLEU trở nên ổn định hơn

```
If next_token == EOS_IDX:
```

```
    Finished = True
```

3. Hàm translate(sentence)

Quy trình của hàm:

1. Tokenize → numericalize → encoder
2. Beam search decoding (beam_size = 5)
3. Lấy beam có log-prob tốt nhất
4. Ghép token → câu tiếng Pháp → trả về kết quả

Beam search giúp tìm câu tối ưu hơn greedy decoding bằng cách xét nhiều nhánh trong không gian tìm kiếm.

4. Đánh giá BLEU Score

```
hyps = []
refs = []

for src_sent, trg_sent in zip(test_en, test_fr):

    # 1. Dự đoán bằng BEAM SEARCH
    hyp = translate_sentence_beam(src_sent, model).split()

    # 2. Tokenize câu French thật (reference)
    ref_tokens = tokenize_fr(trg_sent)

    # 3. Thêm vào list
    hyps.append(hyp)
    refs.append([ref_tokens]) # corpus_bleu yêu cầu dạng [[ref]]

# 4. BLEU
bleu_score = corpus_bleu(refs, hyps)
print(f"BLEU (corpus) on test set: {bleu_score:.4f}")
```

BLEU được tính dựa trên:

- modified n-gram matching (1→4-gram)
- brevity penalty (phạt câu ngắn)
- trung bình BLEU trên toàn bộ test set.

BLEU (corpus) on test set: 0.2729

5. Ví dụ dịch và phân tích lỗi

Để đánh giá chất lượng mô hình một cách toàn diện, nhóm thực hiện tiến hành chọn ra một số câu trong tập test của Multi30K và tiến hành dịch bằng mô hình. Mỗi ví dụ gồm:

- (1) câu gốc tiếng Anh (Source),
- (2) câu tham chiếu tiếng Pháp (Reference),

- (3) câu mô hình sinh ra (Hypothesis),
- (4) phân tích điểm đúng/sai.

Các ví dụ dưới đây được dịch bằng mô hình có Attention + Beam Search (beam = 5).

a. Ví dụ đúng

Ví dụ 1

Source:

Three girls are standing in front of a window of a building.

Reference:

Trois filles se tiennent devant la fenêtre d'un bâtiment

Hypothesis:

trois filles sont debout devant une fenêtre d' un bâtiment.

Phân tích:

- Mô hình dịch chính xác cấu trúc “are standing” → “sont debout”, đúng ngữ pháp tiếng Pháp.
- Tái hiện đầy đủ cụm địa điểm “in front of a window of a building” → “devant une fenêtre d’un bâtiment”.
- Sự khác biệt nhỏ “la fenêtre” → “une fenêtre” không làm thay đổi nghĩa tổng thể.
- Đây là câu đơn giản, độ dài ngắn, giúp mô hình Seq2Seq LSTM duy trì ngữ cảnh tốt và dịch chính xác.

Ví dụ 2

Source:

A person wearing goggles and a hat is sled riding.

Reference:

Une personne portant des lunettes de protection et un chapeau fait de la luge.

Hypothesis:

une personne portant des lunettes et des chapeau fait du vélo.

Phân tích:

- Mô hình dịch đúng phần mô tả chủ thể: “A person wearing goggles and a hat” → “une personne portant des lunettes et des chapeau”, giữ lại cấu trúc mô tả với phân từ hiện tại “portant”.
- Phần mô tả trang phục được tái hiện gần đúng, dù có lỗi nhỏ về số ít/số nhiều (“des chapeau”).
- Sai sót chính nằm ở động từ hành động (“fait du vélo”), nhưng phần danh ngữ đầu câu được mô hình diễn đạt khá chính xác.
- Câu này cho thấy mô hình nắm tốt cấu trúc mô tả ngoại hình nhưng gặp khó ở động từ hành động hiếm trong dữ liệu.

Ví dụ 3

Source:

A girl in a pink coat and flowered goloshes sledding down a hill.

Reference:

Une fille avec une veste rose et des galoches à fleurs descend le long d'une colline en luge..

Hypothesis:

une fille với un manteau rose et des chaussettes de couleur vive sur une colline

Phân tích:

- Mô hình dịch khá chính xác phần mô tả chủ thể: “a girl in a pink coat” → “une fille avec un manteau rose”.
- Cấu trúc mô tả trang phục được giữ đúng dạng: danh từ + tính từ.
- Sai sót chủ yếu nằm ở việc bỏ mất hành động chính, nhưng phần danh ngữ đầu câu được mô hình tái hiện tốt.
- Câu này cho thấy mô hình mạnh ở việc dịch cụm danh ngữ mô tả nhưng yếu ở việc duy trì ngữ cảnh hành động.

Ví dụ 4

Source:

The man is scratching the back of his neck while looking for a book in a book store.

Reference:

L'homme se gratte l'arrière du cou tout en cherchant un livre dans une librairie.

Hypothesis:

l' homme ajuste le dos de son tandis qu' regardant un livre dans un livre .

Phân tích:

- Mô hình dịch đúng được cấu trúc chủ thể “The man” → “l’homme” và giữ lại dạng mô tả hai hành động song song (“tandis que ... regardant”), dù chưa đúng ngữ pháp.
- Phần nghĩa hành động được mô phỏng gần đúng (hành động liên quan đến cơ thể + tìm sách), dù từ vựng chưa chính xác.
- Câu này cho thấy mô hình nắm được khung cấu trúc câu phức nhưng khó xử lý chính xác hành động chi tiết.

Ví dụ 5

Source:

A young man participates in a career while the subject who records it smiles.

Reference:

Un jeune homme participe à une course pendant que le sujet qui le filme sourit

Hypothesis:

un jeune homme travaille dans un sauna tandis que le vent qui sourit

Phân tích:

- Mô hình dịch đúng cụm chủ thể “A young man” → “un jeune homme”, thể hiện khả năng nhận dạng thành phần chính xác.

- Khung cấu trúc hai mệnh đề song song được giữ lại (“tandis que ... sourit”), cho thấy mô hình hiểu dạng câu ghép.
- Sai sót chủ yếu nằm ở lựa chọn từ vựng và gán nghĩa sai; tuy nhiên cấu trúc tổng quát vẫn được mô hình tái hiện đúng.
- Câu này thể hiện mô hình nắm được cấu trúc nhưng chưa nắm vững ngữ nghĩa chi tiết.

b. Ví dụ sai

Ví dụ sai 1

Source:

A man is riding a bicycle on the street.

Reference:

Un homme avec un chapeau orange regardant quelque chose.

Hypothesis:

des policiers en des gilets de sécurité des des des blocs .

Phân tích:

- **Lỗi lệch chủ đề nghiêm trọng (semantic drift / hallucination):** câu nguồn nói về “một người đàn ông đội mũ cam”, nhưng mô hình sinh ra cụm “des policiers...” (cảnh sát) không liên quan.
- **Lỗi lặp cấu trúc/khung câu cố định:** mô hình có xu hướng sinh một mẫu câu quen thuộc (“des policiers en...”) thay vì bám theo nội dung nguồn.
- **Nguyên nhân khả dĩ:** (i) decoder bị “kéo” về các n-gram xuất hiện dày trong dữ liệu hoặc do exposure bias khiến mô hình trôi khỏi quỹ đạo; (ii) khi một vài token đầu bị sai, beam search có thể khuếch đại chuỗi sai vì tiếp tục chọn các token có xác suất cao trong “mẫu câu quen”.
- **Tác động:** nội dung dịch mất hoàn toàn ý chính, không thể chấp nhận ở mức câu.

Ví dụ sai 2

Source:

A Boston Terrier is running on lush green grass in front of a white fence.

Reference:

Un terrier de Boston court sur l'herbe verdoyante devant une clôture blanche.

Hypothesis:

des visiteurs en train de réparer les <unk> métalliques en haut d'entre des poteaux métalliques .

Phân tích:

- **Lỗi OOV / từ hiếm** → <unk>: từ khóa “Boston Terrier” là thực thể hiếm; mô hình thay bằng <unk>. Đây là lỗi phổ biến khi dùng word-level vocab giới hạn 10k.
- **Lỗi thay đổi hành động & ngữ cảnh**: “is running on grass” bị đổi thành “repairing ... poteaux métalliques” (sửa cột kim loại) – lệch ngữ nghĩa mạnh.
- **Lỗi cú pháp**: cụm “en haut d'entre” không tự nhiên trong tiếng Pháp → dấu hiệu mô hình ghép cụm theo thống kê thay vì cấu trúc câu.
- **Nguyên nhân**: thiếu cơ chế subword (BPE) khiến thực thể hiếm bị rơi vào <unk>, từ đó encoder mất tín hiệu quan trọng → decoder dễ “trôi” sang mẫu câu phổ biến.

Ví dụ sai 3

Source:

Five people wearing winter jackets and helmets stand in the snow, with snowmobiles in the background.

Reference:

Cinq personnes ... debout dans la neige, avec des motoneiges en arrière-plan.

Hypothesis:

des policiers en des lunettes de police en des échasses en uniformes rouges , tous des policiers d' art en vestes .

Phân tích:

- Lỗi mất thông tin nhiều thực thể (content omission): “five people / winter jackets / helmets / snow / snowmobiles” gần như biến mất.
- Lỗi thay thế thực thể: “people” → “policiers”, “snowmobiles” → không có; thêm “échasses” (cà kheo) không có trong nguồn.
- Nguyên nhân: câu dài hơn trung bình Multi30K (nhiều cụm danh từ + giới từ). Encoder–decoder LSTM dù có attention vẫn có thể phân tán chú ý khi nhiều chi tiết đồng thời xuất hiện (đặc biệt khi decoding sai từ sớm). Đây là dạng “mất chi tiết” được nêu trong mục phân tích lỗi của đề bài.

Ví dụ sai 4

Source:

A boy in a red uniform is attempting to avoid getting out at home plate, while the catcher in the blue uniform is attempting to catch him.

Reference:

... essaie d'éviter ... tandis que ... essaie de l'attraper.

Hypothesis:

des policiers en des lunettes de sécurité en vestes de police de sécurité en train de travailler sur les <unk> métalliques .

Phân tích:

- **Lỗi cấu trúc quan hệ “while/tandis que” bị sập**: câu nguồn có hai mệnh đề song song, nhưng mô hình không giữ được quan hệ đối lập và hành động “avoid getting out / catch him”.

- **Lỗi dịch sai miền từ vựng:** baseball terms (“home plate”, “catcher”) là từ hiếm → mô hình chuyển sang các từ phổ biến trong miền “security/police”.
- **Nguyên nhân:** vocabulary word-level + thiếu subword + thiếu tín hiệu chuyên biệt cho thuật ngữ thể thao; khi thuật ngữ quan trọng thành <unk>, mô hình không còn “neo” để bám nghĩa.

Ví dụ sai 5

Source:

Six people ride mountain bikes through a jungle environment.

Reference:

Six personnes font du VTT dans un décor de jungle.

Hypothesis:

six policiers prennent des rollers en rollers .

Phân tích:

- **Lỗi dịch sai từ khóa hành động và đối tượng:** “mountain bikes” → “rollers”, “people” → “policiers”.
- **Lỗi lặp:** “en rollers” bị lặp hai lần, cho thấy mô hình dễ mắc **repetition loop** khi decoding.
- **Nguyên nhân:** exposure bias (train dùng teacher forcing, inference dùng token dự đoán) khiến sai một bước kéo theo sai dây chuyền; beam search nếu không có penalty chống lặp sẽ ưu tiên n-gram có xác suất cao và lặp lại.

c. Phân loại lỗi & Giải thích nguyên nhân

Loại lỗi	Tác động	Nguyên nhân chính
OOV / từ hiếm → <unk>	Mất “neo nghĩa”, kéo theo lệch chủ đề	Vocab word-level giới hạn 10k; thực thể/ thuật ngữ hiếm không vào từ điển

Lệch chủ đề / hallucination	Dịch sai ý hoàn toàn	Decoder bị trôi do exposure bias; dữ liệu có các mẫu n-gram thống trị;
Lặp từ/cụm từ (repetition loop)	Câu vô nghĩa, giảm BLEU mạnh	Beam search không có repetition penalty; mô hình tối ưu theo xác suất cục bộ
Bỏ sót chi tiết (content omission)	Dịch thiếu thông tin quan trọng	Câu nhiều chi tiết → attention phân tán; encoder–decoder LSTM vẫn có giới hạn nhớ
Sai quan hệ cú pháp/logic giữa mệnh đề	Câu dịch không đúng logic	Mô hình khó học quan hệ dài; thiếu tín hiệu cú pháp; decoding sai sớm
Sai miền từ vựng (domain drift)	Dịch sai chủ đề	Thuật ngữ hiếm + OOV; mô hình chọn từ phổ biến nhất
Sai hình thái/ngữ pháp Pháp	Giảm độ trôi chảy	Mô hình ghép cụm theo thống kê; thiếu ràng buộc cú pháp

d. Đề xuất cải tiến

Để cải thiện chất lượng BLEU và giảm lỗi dịch, có thể áp dụng các hướng phát triển sau:

Cải tiến về Chiến lược Dữ liệu (Data Strategy Enhancements)

- **Đảo ngược câu nguồn** (Reversing Source Sentences):

Theo nghiên cứu của [Sutskever et al. \(2014\)](#), việc đảo ngược thứ tự chuỗi đầu vào (Ví dụ: "A B C" → "C B A") mang lại hiệu quả bất ngờ cho mô hình Seq2Seq.

- **Cơ chế:** Việc này rút ngắn khoảng cách ("minimal time lag") giữa từ đầu tiên của nguồn và từ đầu tiên của đích. Nhờ đó, Gradient lan truyền ngược (Backpropagation) hiệu quả hơn trong giai đoạn đầu, giúp LSTM thiết lập mối liên kết ngữ nghĩa tốt hơn.

- **Hiệu quả dự kiến:** Giảm chỉ số Perplexity và tăng điểm BLEU đáng kể mà không cần thay đổi kiến trúc mạng.
- **Sử dụng Byte Pair Encoding (BPE):**
Để giải quyết triệt để vấn đề từ hiếm (OOV), phương pháp BPE chia nhỏ từ thành các đơn vị con (subwords). Điều này giúp mô hình có thể "ghép vãn" để dịch những từ chưa từng gặp, thay vì trả về <unk>.

Cải tiến về Kiến trúc Mô hình (Architectural Enhancements)

- **Bi-directional LSTM (LSTM hai chiều):**
Nâng cấp Encoder thành Bi-LSTM([Schuster & Paliwal.1997](#)) để đọc câu văn theo hai hướng: từ trái sang phải và từ phải sang trái. Điều này giúp từ hiện tại không chỉ phụ thuộc vào từ đứng trước mà còn "thấy" được ngữ cảnh của từ đứng sau.
- **Tối ưu hóa cổng LSTM (G²-LSTM):**
Dựa trên nghiên cứu của [Li et al. \(2018\)](#), các cổng (Input/Forget Gate) của LSTM truyền thống thường bão hòa ở giá trị 0.4 – 0.6, dẫn đến việc lọc thông tin không rõ ràng.
 - **Đề xuất:** Sử dụng **Gumbel-Softmax estimator** để ép các giá trị cổng tiệm cận về 0 (đóng hoàn toàn) hoặc 1 (mở hoàn toàn).
 - **Mục tiêu:** Biến LSTM thành một cấu trúc quyết định rời rạc (Discrete decision making), giúp loại bỏ nhiễu hiệu quả hơn. Nghiên cứu cho thấy phương pháp này có thể tăng từ **+0.95 đến +2.22 điểm BLEU** trên tập dữ liệu chuẩn IWSLT14 mà không cần thay đổi kiến trúc Seq2Seq cốt lõi.

Cải tiến về Chiến lược Giải mã (Decoding Strategy)

Hiện tại ,nhóm đã sử dụng Beam Search với beam = 5 . Trong tương lai có thể:

- Thử nghiệm **beam size lớn hơn** (ví dụ 8–10) để xem BLEU có tiếp tục tăng hay không.
- Áp dụng **length normalization và diverse beam search** để:
 - Giảm hiện tượng ưu tiên câu quá ngắn.
 - Sinh ra nhiều phương án dịch đa dạng hơn khi cần.
- Tăng quy mô mô hình (Scaling):
Tăng kích thước Vector Embedding (ví dụ lên 512 hoặc 1024) và số lớp chồng (Stacked LSTM, ví dụ: 2-4 layers) để tăng khả năng biểu diễn (Capacity) của mô hình, giúp học được các mẫu ngôn ngữ phức tạp hơn.

PHẦN VII. KẾT LUẬN, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

7.1 Tổng kết đồ án (Summary)

Trong khuôn khổ đồ án này, nhóm đã nghiên cứu và triển khai thành công một hệ thống dịch máy (Machine Translation) từ tiếng Anh sang tiếng Pháp dựa trên kiến trúc Sequence-to-Sequence (Encoder–Decoder LSTM). Toàn bộ quá trình được xây dựng **từ đầu**, không sử dụng các thư viện seq2seq dựng sẵn, và bao gồm đầy đủ các bước chuẩn mực của một bài toán xử lý ngôn ngữ tự nhiên:

- Tiền xử lý dữ liệu:

Nhóm đã xây dựng quy trình đọc dữ liệu song ngữ Anh–Pháp từ bộ Multi30K, làm sạch và chuẩn hóa văn bản, tách từ (tokenization) bằng spaCy, xây dựng từ điển cho từng ngôn ngữ với giới hạn TOP_K, đồng thời xử lý các token đặc biệt như **<sos>**, **<eos>**, **<unk>**, **<pad>**. Các bước này đảm bảo dữ liệu đầu vào thống nhất và phù hợp để đưa vào mô hình.

- Xây dựng mô hình :

Hệ thống được thiết kế dựa trên kiến trúc Encoder–Decoder sử dụng LSTM, kết hợp với tầng Embedding cho cả tiếng Anh và tiếng Pháp. Ban đầu, nhóm triển khai **mô hình Seq2Seq cơ bản** với context vector cố định. Sau đó, mô hình được

mở rộng với cơ chế Luong Attention và hàm giải mã Beam Search, giúp Decoder có khả năng tập trung vào các phần quan trọng của câu nguồn khi sinh từng từ trong câu đích.

- Huấn luyện và tối ưu:

Quá trình huấn luyện sử dụng PyTorch trên Google Colab, kết hợp các kỹ thuật:

- ☐ Teacher Forcing với tỷ lệ thích hợp.
- ☐ Gradient Clipping để tránh hiện tượng gradient exploding.
- ☐ Early Stopping dựa trên validation loss nhằm hạn chế overfitting.
- ☐ Lưu lại checkpoint mô hình tốt nhất để đánh giá và suy luận.

Nhìn chung, đồ án đã hoàn thành đầy đủ các yêu cầu của đề bài: xử lý dữ liệu, xây dựng mô hình Encoder–Decoder LSTM, cài đặt hàm **translate()**, tính BLEU score và phân tích ví dụ dịch đúng/sai.

7.2 Đánh giá tổng quan kết quả (Overall Evaluation)

Kết quả thực nghiệm cho thấy mô hình hoạt động ổn định và đạt được các mục tiêu đề ra:

- Về mặt kỹ thuật:

Pipeline từ dữ liệu đầu vào đến câu dịch đầu ra vận hành trơn tru, không phát sinh lỗi kích thước tensor hay lỗi bộ nhớ. Các biểu đồ train/validation loss hội tụ theo thời gian, chứng tỏ mô hình đã học được các quy luật thống kê trong dữ liệu ngôn ngữ. Hàm `translate()` hoạt động tốt với các câu tiếng Anh mới, thể hiện đúng luồng: tiền xử lý → Encoder → Decoder (Attention + Beam Search) → câu tiếng Pháp.

- Về chất lượng dịch:

- Mô hình **Seq2Seq cơ bản (không attention, Greedy Decoding)** đạt BLEU khoảng **25** trên tập test Multi30K.
- Khi **bổ sung Luong Attention và sử dụng Beam Search (beam = 5)** trong suy luận, BLEU tăng lên khoảng **28**, cho thấy mô hình cải tiến sinh câu trôi chảy và gần với bản dịch tham chiếu hơn.
- Ở mức độ câu đơn, cấu trúc ngữ pháp phổ biến và từ vựng thông dụng, mô hình cho kết quả khá chính xác. Điều này phù hợp với phạm vi của bộ dữ liệu Multi30K (mô tả ảnh, câu ngắn).

- Hạn chế:

Dù đạt kết quả khả quan, mô hình vẫn tồn tại một số điểm yếu:

- **Giới hạn kiến trúc LSTM và context vector** khiến mô hình gặp khó khăn với các câu dài hoặc chứa nhiều cụm thông tin phức tạp; một số từ ở đầu hoặc giữa câu dễ bị bỏ sót.
- **Vấn đề từ hiếm (OOV)** vẫn xuất hiện do từ điển được xây dựng ở mức từ (word-level) và bị giới hạn về kích thước, dẫn đến việc sử dụng **<unk>** trong những trường hợp mô hình chưa từng thấy.
- Hệ thống chưa khai thác các kiến trúc dịch máy hiện đại hơn như Transformer, vốn đang là chuẩn mực trong các hệ thống dịch thương mại.

7.3 Hướng phát triển (Future Work)

Để nâng cao chất lượng dịch thuật và khắc phục các hạn chế nêu trên, trong tương lai có thể triển khai các hướng phát triển sau:

1. Cải thiện biểu diễn dữ liệu và từ vựng
 - **Áp dụng Byte Pair Encoding (BPE) hoặc các phương pháp subword khác:**
Thay vì tokenization hoàn toàn theo từ, việc chuyển sang mô hình hóa theo subword giúp:
 - Giảm mạnh số lượng từ hiếm và token **<unk>**.
 - Xử lý tốt hơn các từ ghép, biến thể từ và từ có hình thái phong phú trong tiếng Pháp.
Điều này được kỳ vọng sẽ cải thiện BLEU và tăng tính ổn định khi dịch các câu chứa từ vựng ít gặp.
2. Nâng cấp kiến trúc mô hình
 - **Sử dụng Bi-directional LSTM cho Encoder:**
Bi-LSTM cho phép mỗi từ trong câu nguồn nhìn được cả ngữ cảnh phía trước lẫn phía sau, giúp biểu diễn ngữ nghĩa giàu thông tin hơn, đặc biệt có lợi cho cơ chế Attention.
 - **Tăng kích thước và chiều sâu mô hình:**
Tăng số chiều embedding, kích thước hidden state và số lớp LSTM (stacked LSTM) sẽ giúp mô hình có năng lực biểu diễn mạnh hơn, từ đó học được các mẫu câu phức tạp hơn (đi kèm với việc cần thêm tài nguyên tính toán).
 - **Chuyển sang kiến trúc Transformer:**
Trong bước tiếp theo, có thể nghiên cứu xây dựng một mô hình Transformer cho cùng bài toán Anh–Pháp để so sánh. Transformer sử dụng hoàn toàn Self-Attention, hỗ trợ huấn luyện song song và đã chứng minh hiệu quả vượt trội trong dịch máy hiện đại.

3. Tối ưu thêm chiến lược giải mã (Decoding Strategy)

- Hiện tại, mô hình đã sử dụng **Beam Search** với **beam = 5**. Trong tương lai có thể:
 - Thử nghiệm các giá trị beam size khác (ví dụ 8 hoặc 10) và áp dụng **length normalization** để cân bằng giữa câu ngắn và câu dài.
 - Khảo sát các biến thể như **diverse beam search** nhằm tạo ra nhiều phương án dịch đa dạng hơn trong những ứng dụng cần gợi ý nhiều câu dịch thay vì chỉ một kết quả.

Nhìn chung, đồ án đã hoàn thành mục tiêu xây dựng một hệ thống dịch máy Anh–Pháp dựa trên Encoder–Decoder LSTM, đồng thời bước đầu thử nghiệm các kỹ thuật cải tiến như Attention và Beam Search. Các hướng phát triển nêu trên là cơ sở để mở rộng đồ án thành một hệ thống dịch máy mạnh hơn và tiệm cận hơn với các mô hình dịch hiện đại trong thực tế.

PHẦN VIII. TÀI LIỆU THAM KHẢO

- [1] I. Sutskever, O. Vinyals and Q. V. Le, “Sequence to Sequence Learning with Neural Networks,” in *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, Montreal, Canada, 2014. [arXiv+1](#)
- [2] D. Bahdanau, K. Cho and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *Proc. 3rd Int. Conf. on Learning Representations (ICLR)*, 2015. [arXiv](#)
- [3] M.-T. Luong, H. Pham and C. D. Manning, “Effective Approaches to Attention-based Neural Machine Translation,” in *Proc. 2015 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, Lisbon, Portugal, 2015, pp. 1412–1421. [arXiv+1](#)
- [4] K. Papineni, S. Roukos, T. Ward and W.-J. Zhu, “BLEU: A Method for Automatic Evaluation of Machine Translation,” in *Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, Philadelphia, PA, USA, 2002, pp. 311–318. [ACL Anthology+1](#)

- [5] D. Elliott, S. Frank, K. Sima'an and L. Specia, "Multi30K: Multilingual English–German Image Descriptions," in *Proc. 5th Workshop on Vision and Language*, Berlin, Germany, 2016, pp. 70–74. [arXiv](#)
- [6] R. Sennrich, B. Haddow and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Berlin, Germany, 2016, pp. 1715–1725. [ACL Anthology+1](#)
- [7] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, Long Beach, CA, USA, 2017, pp. 5998–6008. [arXiv+1](#)
- [8] M. Honnibal, I. Montani, S. Van Landeghem and A. Boyd, "spaCy: Industrial-strength Natural Language Processing in Python," 2020. [Online]. Available: <https://spacy.io> [spacy.io+1](#)

PHỤ LỤC

A. Mã nguồn chính (Source Code Structure)

```
TOP_K = 10000
MIN_FREQ = 1
SPECIALS = ["<pad>", "<sos>", "<eos>", "<unk>"]

def build_vocab_from_tokenized(sentences_tok, top_k=TOP_K,
min_freq=MIN_FREQ):
    counter = Counter()
    for tokens in sentences_tok:
        counter.update(tokens)
    most = [w for w,c in counter.most_common(top_k) if c >= min_freq]
    stoi = {tok: idx for idx, tok in enumerate(SPECIALS)}
    for w in most:
        if w not in stoi:
```

```
    stoi[w] = len(stoi)
    itos = {idx: tok for tok, idx in stoi.items()}
    return stoi, itos
```

```
class TranslationDataset(Dataset):
    def __init__(self, src_list, trg_list, src_tokenizer, trg_tokenizer, src_stoi,
trg_stoi):
        assert len(src_list) == len(trg_list)
        self.src = src_list
        self.trg = trg_list
        self.src_tokenizer = src_tokenizer
        self.trg_tokenizer = trg_tokenizer
        self.src_stoi = src_stoi
        self.trg_stoi = trg_stoi

    def __len__(self):
        return len(self.src)

    def __getitem__(self, idx):
        src_tokens = self.src_tokenizer(self.src[idx])
        trg_tokens = self.trg_tokenizer(self.trg[idx])
        return src_tokens, trg_tokens
```

```
def collate_fn(batch):
    src_batch, trg_batch = [], []
    for src_tokens, trg_tokens in batch:
        src_batch.append(numericalize(src_tokens, stoi_en, add_sos_eos=False))
        trg_batch.append(numericalize(trg_tokens, stoi_fr, add_sos_eos=True))

    src_lengths = [len(s) for s in src_batch]
    trg_lengths = [len(t) for t in trg_batch]

    sorted_idx = sorted(range(len(src_lengths)), key=lambda i: src_lengths[i],
reverse=True)
    src_batch = [torch.tensor(src_batch[i], dtype=torch.long) for i in sorted_idx]
    trg_batch = [torch.tensor(trg_batch[i], dtype=torch.long) for i in sorted_idx]
```

```

src_lengths = [src_lengths[i] for i in sorted_idx]
trg_lengths = [trg_lengths[i] for i in sorted_idx]

src_padded = pad_sequence(src_batch, batch_first=True,
padding_value=stoi_en["<pad>"])
trg_padded = pad_sequence(trg_batch, batch_first=True,
padding_value=stoi_fr["<pad>"])

return (src_padded.to(DEVICE),
        torch.tensor(src_lengths, dtype=torch.long).to(DEVICE),
        trg_padded.to(DEVICE),
        torch.tensor(trg_lengths, dtype=torch.long).to(DEVICE))

```

```

class LuongAttention(nn.Module):
    def __init__(self, hid_dim):
        super().__init__()
        self.attn = nn.Linear(hid_dim, hid_dim, bias=False)
    def forward(self, decoder_hidden, encoder_outputs, mask):
        dec_hidden = self.attn(decoder_hidden).unsqueeze(2)
        energy = torch.bmm(encoder_outputs, dec_hidden).squeeze(2)
        energy = energy.masked_fill(mask == 0, -1e10)
        attn_weights = torch.softmax(energy, dim=1)
        return attn_weights

```

```

class Encoder(nn.Module):
    def __init__(self, input_dim, emb_dim, hid_dim, n_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim,
padding_idx=stoi_en["<pad>"])
        self.rnn = nn.LSTM(emb_dim, hid_dim, n_layers, dropout=dropout,
batch_first=True)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, src_lengths):
        # src: [batch, src_len]
        embedded = self.dropout(self.embedding(src)) # [batch, src_len, emb]
        packed = pack_padded_sequence(embedded, src_lengths.cpu(),

```

```

batch_first=True, enforce_sorted=True)
    packed_outputs, (hidden, cell) = self.rnn(packed)
    outputs, _ = pad_packed_sequence(packed_outputs, batch_first=True) #
[batch, src_len, hid]
    return outputs, hidden, cell

```

```

class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, hid_dim, n_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(output_dim, emb_dim,
padding_idx=PAD_IDX)
        self.rnn = nn.LSTM(emb_dim + hid_dim, hid_dim, n_layers,
dropout=dropout, batch_first=True)
        self.fc_out = nn.Linear(hid_dim * 2, output_dim)
        self.dropout = nn.Dropout(dropout)
        self.attention = LuongAttention(hid_dim)

    def forward(self, input_step, hidden, cell, encoder_outputs, mask):
        # input_step: [batch] token ids
        input_step = input_step.unsqueeze(1) # [batch,1]
        embedded = self.dropout(self.embedding(input_step)) # [batch,1,emb_dim]

        dec_hidden = hidden[-1] # last layer hidden: [batch, hid_dim]
        attn_weights = self.attention(dec_hidden, encoder_outputs, mask) # [batch,
src_len]
        attn_weights_unsq = attn_weights.unsqueeze(1) # [batch,1,src_len]

        context = torch.bmm(attn_weights_unsq, encoder_outputs) #
[batch,1,hid_dim]
        rnn_input = torch.cat((embedded, context), dim=2) # [batch,1,emb+hid]

        output, (hidden, cell) = self.rnn(rnn_input, (hidden, cell)) # output:
[batch,1,hid]
        output = output.squeeze(1) # [batch,hid]
        context = context.squeeze(1) # [batch,hid]

```

```

        pred = self.fc_out(torch.cat((output, context), dim=1)) # [batch,
output_dim]
        return pred, hidden, cell, attn_weights # attn_weights: [batch, src_len]

```

```

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device, trg_pad_idx=PAD_IDX):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device
        self.trg_pad_idx = trg_pad_idx

    def create_mask(self, src):
        return (src != stoi_en["<pad>"]).to(self.device) # [batch, src_len]

    def forward(self, src, src_lengths, trg, teacher_forcing_ratio=0.5):
        batch_size = src.size(0)
        trg_len = trg.size(1)
        trg_vocab_size = self.decoder.fc_out.out_features

        outputs = torch.zeros(batch_size, trg_len, trg_vocab_size).to(self.device)
        encoder_outputs, hidden, cell = self.encoder(src, src_lengths)
        mask = self.create_mask(src)

        input_token = trg[:, 0] # <sos>
        for t in range(1, trg_len):
            output, hidden, cell, _ = self.decoder(input_token, hidden, cell,
encoder_outputs, mask)
            outputs[:, t, :] = output
            teacher_force = random.random() < teacher_forcing_ratio
            top1 = output.argmax(1)
            input_token = trg[:, t] if teacher_force else top1

        return outputs

```

```

N_EPOCHS = 20
CLIP = 1.0
patience = 3
best_valid_loss = float('inf')
stale = 0
train_losses, valid_losses = [], []

def epoch_time(start_time, end_time):
    elapsed = end_time - start_time
    m = int(elapsed/60)
    s = int(elapsed - m*60)
    return m, s

for epoch in range(N_EPOCHS):
    start_time = time.time()
    model.train()
    train_loss = 0

    for src, src_lens, trg, trg_lens in train_loader:
        src, src_lens, trg = src.to(DEVICE), src_lens.to(DEVICE), trg.to(DEVICE)
        optimizer.zero_grad()
        output = model(src, src_lens, trg, teacher_forcing_ratio=0.5)
        output_dim = output.shape[-1]
        output = output[:,1:,:].reshape(-1, output_dim)
        trg_y = trg[:,1:,:].reshape(-1)
        loss = criterion(output, trg_y)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), CLIP)
        optimizer.step()
        train_loss += loss.item()

    model.eval()
    valid_loss = 0
    with torch.no_grad():
        for src, src_lens, trg, trg_lens in val_loader:
            src, src_lens, trg = src.to(DEVICE), src_lens.to(DEVICE),

```



```

trg.to(DEVICE)
    output = model(src, src_lens, trg, teacher_forcing_ratio=0)
    output_dim = output.shape[-1]
    output = output[:,1:,:].reshape(-1, output_dim)
    trg_y = trg[:,1:].reshape(-1)
    loss = criterion(output, trg_y)
    valid_loss += loss.item()

train_loss = train_loss / len(train_loader)
valid_loss = valid_loss / len(val_loader)
train_losses.append(train_loss)
valid_losses.append(valid_loss)

end_time = time.time()
epoch_m, epoch_s = epoch_time(start_time, end_time)
print(f'Epoch {epoch+1} | Time {epoch_m}m {epoch_s}s | Train Loss:
{train_loss:.4f} | Val Loss: {valid_loss:.4f}')

# early stopping & save best
if valid_loss < best_valid_loss:
    best_valid_loss = valid_loss
    best_model_state = model.state_dict()
    stale = 0
else:
    stale += 1
    if stale >= patience:
        print("Early stopping triggered.")
        break

```

```

def translate_sentence_beam(sentence, model, beam_width=5, max_len=50):
    model.eval()

    # Chuẩn hoá input
    tokens = tokenize_en(sentence)

```

```

src_indices = [stoi_en.get(t, UNK_IDX) for t in tokens]
src_tensor = torch.tensor(src_indices,
dtype=torch.long).unsqueeze(0).to(DEVICE)
src_len = torch.tensor([len(src_indices)], dtype=torch.long).to(DEVICE)

# Encoder
with torch.no_grad():
    encoder_outputs, hidden, cell = model.encoder(src_tensor, src_len)
    mask = model.create_mask(src_tensor)

# Beam: mỗi phần tử = (sequence, hidden, cell, score)
beams = [( [SOS_IDX], hidden, cell, 0 )] # log-prob score

for _ in range(max_len):
    new_beams = []

    for seq, h, c, score in beams:

        last_token = seq[-1]

        # Nếu đã kết thúc câu
        if last_token == EOS_IDX:
            new_beams.append((seq, h, c, score))
            continue

        input_token = torch.tensor([last_token], dtype=torch.long).to(DEVICE)

        with torch.no_grad():
            output, h_new, c_new, _ = model.decoder(
                input_token, h, c, encoder_outputs, mask
            )

```

```

log_probs = torch.log_softmax(output, dim=1) # [1, vocab]
topk_log_probs, topk_ids = torch.topk(log_probs, beam_width)

# Thêm vào beam mới
for i in range(beam_width):
    new_seq = seq + [topk_ids[0][i].item()]
    new_score = score + topk_log_probs[0][i].item()
    new_beams.append((new_seq, h_new, c_new, new_score))

# Giữ lại beam tốt nhất
new_beams = sorted(new_beams, key=lambda x: x[3], reverse=True)
beams = new_beams[:beam_width]

# Nếu toàn bộ beam kết thúc EOS → stop
if all(b[0][-1] == EOS_IDX for b in beams):
    break

# Beam tốt nhất
best_seq = beams[0][0][1:] # bỏ SOS

# Dừng ở EOS
final_words = []
for idx in best_seq:
    if idx == EOS_IDX:
        break
    final_words.append(itos_fr[idx])

return " ".join(final_words)

```

B. Toàn bộ Notebook

Mã nguồn đầy đủ và quy trình thực nghiệm chi tiết được lưu trữ tại Google Colab Notebook dưới đây:

- **Link:** [NLP_Attention_Beam.ipynb](#)

- **Lưu ý:** Để chạy lại notebook, vui lòng tải bộ dữ liệu vào thư mục gốc của Drive hoặc upload trực tiếp lên session.
- **Link folder** làm việc của nhóm : [NLP_SGU_2025](#)

C. Log Training (Nhật ký huấn luyện)

Trích xuất mẫu 5 epoch đầu tiên và cuối cùng:

Epoch	Train Loss	Val Loss	Time (m:s)	Ghi chú
1	4.5686	4.2521	1:35	Khởi đầu, Loss giảm nhanh
2	3.812	3.4034	1:34	
...	...	...	...	
13	1.2748	3.1560	1:34	Có dấu hiệu Overfitting nhẹ
14	1.1901	3.2270	1:34	Dừng huấn luyện (Early Stopping)

D. Checkpoint mô hình

File trọng số mô hình tốt nhất (**best_model.pth**) đã được lưu trữ và có thể tải về để chạy suy diễn (inference) ngay lập tức:

- **Đường dẫn:** [best_model.pth](#)
- **Kích thước:** ~92MB.
- **Nội dung:** Trọng số của Encoder–Decoder LSTM (kèm Attention) tại epoch tốt nhất
- **Mục đích:** Cho phép tải và chạy suy luận trực tiếp mà không cần huấn luyện lại mô hình

QUY TRÌNH KIỂM THỬ (INFERENCE) MÔ HÌNH DỊCH MÁY ANH-PHÁP

Notebook kiểm thử được thiết kế **độc lập** với quá trình huấn luyện, phục vụ mục đích **nạp lại mô hình đã huấn luyện và đánh giá khả năng suy luận (inference)** trên các tập dữ liệu validation và test.

Cách chạy

Bước 1: Truy cập notebook kiểm thử

Người dùng mở file notebook (ở folder [NLP_SGU_2025](#))



Người dùng mở file notebook [Test.ipynb](#) kiểm thử nằm trong thư mục **NLP_SGU_2025**.

Notebook này được xây dựng riêng cho mục đích **inference**, không thực hiện huấn luyện lại mô hình.

Bước 2: Kết nối Google Colab với Google Drive

Trong notebook, chạy cell đầu tiên để mount Google Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

Sau khi xác nhận, toàn bộ file trên Google Drive sẽ được truy cập tại:

`/content/drive`

Bước 3: Cài đặt và import thư viện cần thiết

Người dùng chạy các cell cài đặt và import thư viện.

Các cell này **chỉ cần chạy một lần duy nhất**:

```
!pip install torch torchtext spacy nltk
!python -m spacy download en_core_web_sm
!python -m spacy download fr_core_news_sm
!pip install gdown
```

Import các thư viện sử dụng trong notebook:

```
import os
import random
import time
from collections import Counter
import spacy
import nltk
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence,
pad_packed_sequence
```

```
from nltk.translate.bleu_score import corpus_bleu
import requests
import gdown
```

Bước 4: Đọc dữ liệu trực tiếp đã lưu ở Google Drive tại nơi làm việc đồ án

Dữ liệu huấn luyện, validation và test được lưu trên Google Drive dưới dạng các file văn bản.

Notebook sử dụng API HTTP để đọc trực tiếp nội dung các file này.

```
def read_lines_from_url(url):
    file_id = url.split('/d/')[1].split('/')[0]
    download_url = f'https://drive.google.com/uc?export=download&id={file_id}'
    response = requests.get(download_url)
    if response.status_code == 200:
        return [line.strip() for line in response.text.splitlines()]
    else:
        print(f'Lỗi không đọc được link: {url}')
    return []

train_en =
read_lines_from_url("https://drive.google.com/file/d/1QytxL95PqumFwCpxrmE9WgvowYrdNPfk/view?usp=drive_link")

train_fr =
read_lines_from_url("https://drive.google.com/file/d/1uRbRaAOEG22fHx5-jl9WWlfdwr8cbAX/view?usp=drive_link")
```

```
val_en =  
read_lines_from_url("https://drive.google.com/file/d/1LQ99amqqy4dK5ivU4zge  
McIBKEYWhXGx/view?usp=drive_link")  
  
val_fr =  
read_lines_from_url("https://drive.google.com/file/d/1i3Q7ct5Bj_mdcVzfiH4iy7  
O8d56dC05P/view?usp=drive_link")  
  
test_fr =  
read_lines_from_url("https://drive.google.com/file/d/1OOROf6d5scqpzb5TnqT6  
W9Yx6mGycC2A/view?usp=drive_link")  
  
test_en =  
read_lines_from_url("https://drive.google.com/file/d/1gRw5GsP8q-wiDxvoKRt  
GGKLVtR_tkx1V/view?usp=drive_link")
```

Bước 5: Tiếp tục chạy các cell bên dưới để khởi tạo lại như lúc huấn luyện

5.1 Tokenization

Notebook sử dụng **spaCy** để tách từ cho tiếng Anh và tiếng Pháp:

```
nlp_en = spacy.load("en_core_web_sm")  
nlp_fr = spacy.load("fr_core_news_sm")  
  
def tokenize_en(text):  
    return [tok.text.lower() for tok in nlp_en.tokenizer(text)]  
  
def tokenize_fr(text):  
    return [tok.text.lower() for tok in nlp_fr.tokenizer(text)]
```

5.2 Xây dựng từ vựng (Vocabulary)


```

TOP_K = 10000
MIN_FREQ = 1
SPECIALS = ["<pad>", "<sos>", "<eos>", "<unk>"]

def build_vocab_from_tokenized(sentences_tok, top_k=TOP_K,
min_freq=MIN_FREQ):
    counter = Counter()
    for tokens in sentences_tok:
        counter.update(tokens)
    most = [w for w,c in counter.most_common(top_k) if c >= min_freq]
    stoi = {tok: idx for idx, tok in enumerate(SPECIALS)}
    for w in most:
        if w not in stoi:
            stoi[w] = len(stoi)
    itos = {idx: tok for tok, idx in stoi.items()}
    return stoi, itos

```

```

#Tiến hành xây dựng từ vựng
train_en_tok = [tokenize_en(s) for s in train_en]
train_fr_tok = [tokenize_fr(s) for s in train_fr]

stoi_en, itos_en = build_vocab_from_tokenized(train_en_tok)
stoi_fr, itos_fr = build_vocab_from_tokenized(train_fr_tok)
# xác định chỉ số đặc biệt
PAD_IDX = stoi_fr["<pad>"]
SOS_IDX = stoi_fr["<sos>"]
EOS_IDX = stoi_fr["<eos>"]
UNK_IDX = stoi_fr["<unk>"]

```

Bước 6: Khởi tạo lại kiến trúc mô hình

Notebook định nghĩa lại đầy đủ các thành phần của mô hình giống hết giai đoạn huấn luyện:

- Luong Attention
- Encoder (LSTM)
- Decoder (LSTM + Attention)
- Mô hình Seq2Seq

(Phần code định nghĩa lớp được giữ nguyên như trong quá trình huấn luyện.)

```
# -----
# Model: Luong Attention + Encoder/Decoder
# -----
class LuongAttention(nn.Module):

    """
    Luong Attention:
    - Tính trọng số chú ý (attention weights) giữa hidden state của Decoder và
    toàn bộ hidden states của Encoder.
    - Giúp Decoder tập trung vào các từ quan trọng trong câu nguồn khi sinh ra
    từng từ đích.
    """

    def __init__(self, hid_dim):
        super().__init__()
        self.attn = nn.Linear(hid_dim, hid_dim, bias=False)

    def forward(self, decoder_hidden, encoder_outputs, mask):
        # decoder_hidden: [batch, hid]
        # encoder_outputs: [batch, src_len, hid]
        dec_hidden = self.attn(decoder_hidden).unsqueeze(2)
        energy = torch.bmm(encoder_outputs, dec_hidden).squeeze(2)
        energy = energy.masked_fill(mask == 0, -1e10)
        attn_weights = torch.softmax(energy, dim=1)
        return attn_weights # [batch, src_len]

class Encoder(nn.Module):
```

```
"""
```

Encoder:

- Nhận câu nguồn (tiếng Anh) dưới dạng chuỗi chỉ số từ.
- Sử dụng Embedding + LSTM nhiều tầng để mã hoá câu nguồn thành chuỗi vector ngữ nghĩa (encoder_outputs).
- Trả về:
 - + encoder_outputs: biểu diễn ngữ cảnh của từng từ nguồn
 - + hidden, cell: trạng thái cuối của LSTM, dùng để khởi tạo Decoder

```
"""
```

```
def __init__(self, input_dim, emb_dim, hid_dim, n_layers=2, dropout=0.3):
    super().__init__()
    self.embedding = nn.Embedding(input_dim, emb_dim,
padding_idx=stoi_en["<pad>"])
    self.rnn = nn.LSTM(
        emb_dim, hid_dim,
        n_layers,
        dropout=dropout,
        batch_first=True
    )
    self.dropout = nn.Dropout(dropout)

def forward(self, src, src_lengths):
    embedded = self.dropout(self.embedding(src))
    packed = pack_padded_sequence(
        embedded,
        src_lengths.cpu(),
        batch_first=True,
        enforce_sorted=True
    )
    packed_outputs, (hidden, cell) = self.rnn(packed)
    outputs, _ = pad_packed_sequence(packed_outputs, batch_first=True)
    return outputs, hidden, cell
```

```
class Decoder(nn.Module):
```

```
"""
```

Decoder:

- Sinh từng từ của câu đích (tiếng Pháp) theo từng bước thời gian.
- Ở mỗi bước:

- + Nhận từ trước đó
- + Áp dụng Luong Attention để lấy context từ Encoder
- + Kết hợp embedding + context đưa vào LSTM
- + Dự đoán từ tiếp theo

"""

```
def __init__(self, output_dim, emb_dim, hid_dim, n_layers=2, dropout=0.3):
    super().__init__()
    self.embedding = nn.Embedding(output_dim, emb_dim,
padding_idx=PAD_IDX)
    self.rnn = nn.LSTM(
        emb_dim + hid_dim,
        hid_dim,
        n_layers,
        dropout=dropout,
        batch_first=True
    )
    self.fc_out = nn.Linear(hid_dim * 2, output_dim)
    self.dropout = nn.Dropout(dropout)
    self.attention = LuongAttention(hid_dim)

def forward(self, input_step, hidden, cell, encoder_outputs, mask):
    input_step = input_step.unsqueeze(1)
    embedded = self.dropout(self.embedding(input_step))

    dec_hidden = hidden[-1]
    attn_weights = self.attention(dec_hidden, encoder_outputs, mask)
    attn_weights_unsq = attn_weights.unsqueeze(1)

    context = torch.bmm(attn_weights_unsq, encoder_outputs)
    rnn_input = torch.cat((embedded, context), dim=2)

    output, (hidden, cell) = self.rnn(rnn_input, (hidden, cell))
    output = output.squeeze(1)
    context = context.squeeze(1)

    pred = self.fc_out(torch.cat((output, context), dim=1))
    return pred, hidden, cell, attn_weights

class Seq2Seq(nn.Module):
```

```
"""
```

Seq2Seq:

- Kết hợp Encoder và Decoder thành mô hình dịch hoàn chỉnh.
- Trong quá trình huấn luyện:
 - + Encoder mã hoá câu nguồn
 - + Decoder sinh câu đích với teacher forcing
- Được dùng cho cả train và validation

```
"""
```

```
def __init__(self, encoder, decoder, device, trg_pad_idx=PAD_IDX):
    super().__init__()
    self.encoder = encoder
    self.decoder = decoder
    self.device = device
    self.trg_pad_idx = trg_pad_idx
```

```
def create_mask(self, src):
    return (src != stoi_en["<pad>"]).to(self.device)
```

```
def forward(self, src, src_lengths, trg, teacher_forcing_ratio=0.5):
    batch_size = src.size(0)
    trg_len = trg.size(1)
    trg_vocab_size = self.decoder.fc_out.out_features
```

```
    outputs = torch.zeros(
        batch_size,
        trg_len,
        trg_vocab_size
    ).to(self.device)
```

```
    encoder_outputs, hidden, cell = self.encoder(src, src_lengths)
    mask = self.create_mask(src)
```

```
    input_token = trg[:, 0] # <sos>
    for t in range(1, trg_len):
        output, hidden, cell, _ = self.decoder(
            input_token,
            hidden,
            cell,
            encoder_outputs,
```

```

        mask
    )
    outputs[:, t, :] = output
    teacher_force = random.random() < teacher_forcing_ratio
    top1 = output.argmax(1)
    input_token = trg[:, t] if teacher_force else top1

return outputs

```

Cấu hình mô hình

```

INPUT_DIM = len(stoi_en)
OUTPUT_DIM = len(stoi_fr)
ENC_EMB_DIM = 256
DEC_EMB_DIM = 256
HID_DIM = 512
N_LAYERS = 2
DROPOUT = 0.3

enc = Encoder(INPUT_DIM, ENC_EMB_DIM, HID_DIM,
              n_layers=N_LAYERS, dropout=DROPOUT)

dec = Decoder(OUTPUT_DIM, DEC_EMB_DIM, HID_DIM,
              n_layers=N_LAYERS, dropout=DROPOUT)

model = Seq2Seq(enc, dec, DEVICE, trg_pad_idx=PAD_IDX).to(DEVICE)

```

Bước 7: Định nghĩa hàm dịch với Beam Search

Notebook sử dụng **Beam Search** để suy luận bản dịch, giúp cải thiện chất lượng so với greedy decoding.

```

def translate_sentence_beam(sentence, model, beam_width=5, max_len=50):
    model.eval()

    # Chuẩn hoá input
    tokens = tokenize_en(sentence)
    src_indices = [stoi_en.get(t, UNK_IDX) for t in tokens]
    src_tensor = torch.tensor(src_indices,
dtype=torch.long).unsqueeze(0).to(DEVICE)
    src_len = torch.tensor([len(src_indices)], dtype=torch.long).to(DEVICE)

    # Encoder
    with torch.no_grad():
        encoder_outputs, hidden, cell = model.encoder(src_tensor, src_len)
        mask = model.create_mask(src_tensor)

    # Beam: mỗi phần tử = (sequence, hidden, cell, score)
    beams = [( [SOS_IDX], hidden, cell, 0 )] # log-prob score

    for _ in range(max_len):
        new_beams = []

        for seq, h, c, score in beams:

            last_token = seq[-1]

            # Nếu đã kết thúc câu
            if last_token == EOS_IDX:
                new_beams.append((seq, h, c, score))
                continue

            input_token = torch.tensor([last_token], dtype=torch.long).to(DEVICE)

```

```

with torch.no_grad():
    output, h_new, c_new, _ = model.decoder(
        input_token, h, c, encoder_outputs, mask
    )

log_probs = torch.log_softmax(output, dim=1) # [1, vocab]
topk_log_probs, topk_ids = torch.topk(log_probs, beam_width)

# Thêm vào beam mới
for i in range(beam_width):
    new_seq = seq + [topk_ids[0][i].item()]
    new_score = score + topk_log_probs[0][i].item()
    new_beams.append((new_seq, h_new, c_new, new_score))

# Giữ lại beam tốt nhất
new_beams = sorted(new_beams, key=lambda x: x[3], reverse=True)
beams = new_beams[:beam_width]

# Nếu toàn bộ beam kết thúc EOS → stop
if all(b[0][-1] == EOS_IDX for b in beams):
    break

# Beam tốt nhất
best_seq = beams[0][0][1:] # bỏ SOS

# Dừng ở EOS
final_words = []
for idx in best_seq:
    if idx == EOS_IDX:
        break
    final_words.append(itos_fr[idx])

return " ".join(final_words)

```


Hàm trả về **một chuỗi tiếng Pháp hoàn chỉnh** tương ứng với câu tiếng Anh đầu vào.

Bước 8: Load trọng số mô hình đã huấn luyện

Mô hình đã huấn luyện được lưu dưới dạng state_dict tại Google Drive.
Notebook tiến hành nạp lại trọng số:

```
!gdown --id 1a3cSV-lcf4dIEpzRu_o1sKp1DIkt2QNA -O best_model.pth

model.load_state_dict(torch.load('/content/best_model.pth',
                                map_location=DEVICE))

model.to(DEVICE)
```

Bước 9: Chạy toàn bộ notebook

Sau khi hoàn tất các bước trên, người dùng chỉ cần chọn:

Runtime → Run all

Notebook sẽ được thực thi tuần tự từ trên xuống dưới và cho phép:

- Kiểm tra kết quả dịch trên tập validation/test
- Đánh giá định tính chất lượng suy luận của mô hình